

Control Hijacking: Defenses

CSCI 4130: Introduction to Cyber Security

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Memory Corruption Attacks

- The goal of attackers who attempt to corrupt the memory of a process **usually** is to **take control of it**
 - Run with the privilege of the process
 - Execute attacker-specified code
- Control-hijacking attacks are made possible by
 - Mixing **data** and **control (code)** in memory
 - Allowing a **user** to **overwrite control data**
 - Usually a pointer for future code execution

Reasoning about Memory Safety

- Memory Safety
 - No access to “undefined” memory
 - “Undefined” is with respect to the semantics of the programming language
 - Accesses may include read, write and execute
- How can we have confidence that the code we write executes in a safe fashion?
 - Build confidence by a module-by-module analysis

Reasoning about Memory Safety (Cont.)

- Reason about the memory properties of using a function or executing a statement
 - **Preconditions**: what must hold for the function or the statement to operate correctly
 - **Postconditions**: what holds after the function returns or the statement is executed
 - The postcondition of Stmt#1 should logically imply the precondition of Stmt#2
 - Invariants: the conditions that always hold at a given point in a function

Preconditions & Postconditions

- **Precondition**

- An assertion that must hold about the inputs of a function $f()$ to make it work correctly
- The caller must meet the precondition
- The callee $f()$ may assume that the precondition is met

- **Postcondition**

- An assertion that holds after a function $f()$ returns
- The caller may assume that the postcondition holds
- The callee must make sure that the postcondition holds

Reasoning about Memory Safety (Cont.)

```
/* requires: p != NULL (and p is a valid pointer) */
```

```
int deref(int *p) {  
    return *p;  
}
```

- **Precondition:** what must hold for the function to operate correctly
 - Needs to be written in a way that a person calling the function knows how to evaluate

Reasoning about Memory Safety (Cont.)

```
/* eusures: retval != NULL (and is a valid pointer) */
```

```
void *my_malloc(size_t n) {  
    void *p = malloc(n);  
    if (!p) { perror("malloc"); exit(1); }  
    return p;  
}
```

- **Postcondition:** what the function promises to hold after it returns
 - Also needs to be written in a way that a person using the function knows how to make use of the function

Reasoning about Memory Safety (Cont.)

```
int count(int a[], size_t n, int val) {  
    int c = 0;  
    for (size_t i=0; i<n; i++) {  
        /* requires a != NULL && 0 <= i && i < size(a) */  
        if (a[i] == val) c++;  
    }  
    return c;  
}
```

We assume a never changes

- General correctness proof strategy for memory safety
 - Identify each memory access code point
 - Write down precondition it requires
 - Propagate requirement up to the beginning of the function

Reasoning about Memory Safety (Cont.)

```
/* requires a != NULL */
int count(int a[], size_t n, int val) {
    int c = 0;
    for (size_t i=0; i<n; i++) {
        /* requires 0 <= i && i < size(a) */
        if (a[i] == val) c++;
    }
    return c;
}
```

0 <= i ?

- General correctness proof strategy for memory safety
 - Identify each memory access code point
 - Write down precondition it requires
 - Propagate requirement up to the beginning of the function

Reasoning about Memory Safety (Cont.)

```
/* requires a != NULL && n <= size(a) */  
int count(int a[], size_t n, int val) {  
    int c = 0;  
    for (size_t i=0; i<n; i++) {  
        /* invariant? i < n && n <= size(a) */  
        /* requires i < size(a) */  
        if (a[i] == val) c++;  
    }  
    return c;  
}
```

$i < n$

$i < \text{size}(a)$?

n never changes

/* invariant: $a \neq \text{NULL} \ \&\& \ 0 \leq i \ \&\& \ i < n \ \&\& \ n \leq \text{size}(a)$ */

- A more complicated loop might require us to use induction
 - Base case: first entrance into loop
 - Induction: show that postcondition of the last statement in loop plus loop test condition, implies invariant

Class Practice

- What is the post condition of this function?

```
/* eusures: ??? */
```

```
int hash(char *s) {  
    int h = 23;  
    while (*s)  
        h = 251*h + (*s++) + 3;  
    return h % N;  
}
```

Memory Corruption Defenses

- Prevention
 - Write bug-free program (can be difficult)
 - Rewrite program in a memory-safe language that automatically checks boundary and does not allow pointer arithmetic
 - Analyze the program for potential vulnerabilities before execution
 - Prevent attack code execution
 - Make exploitation harder

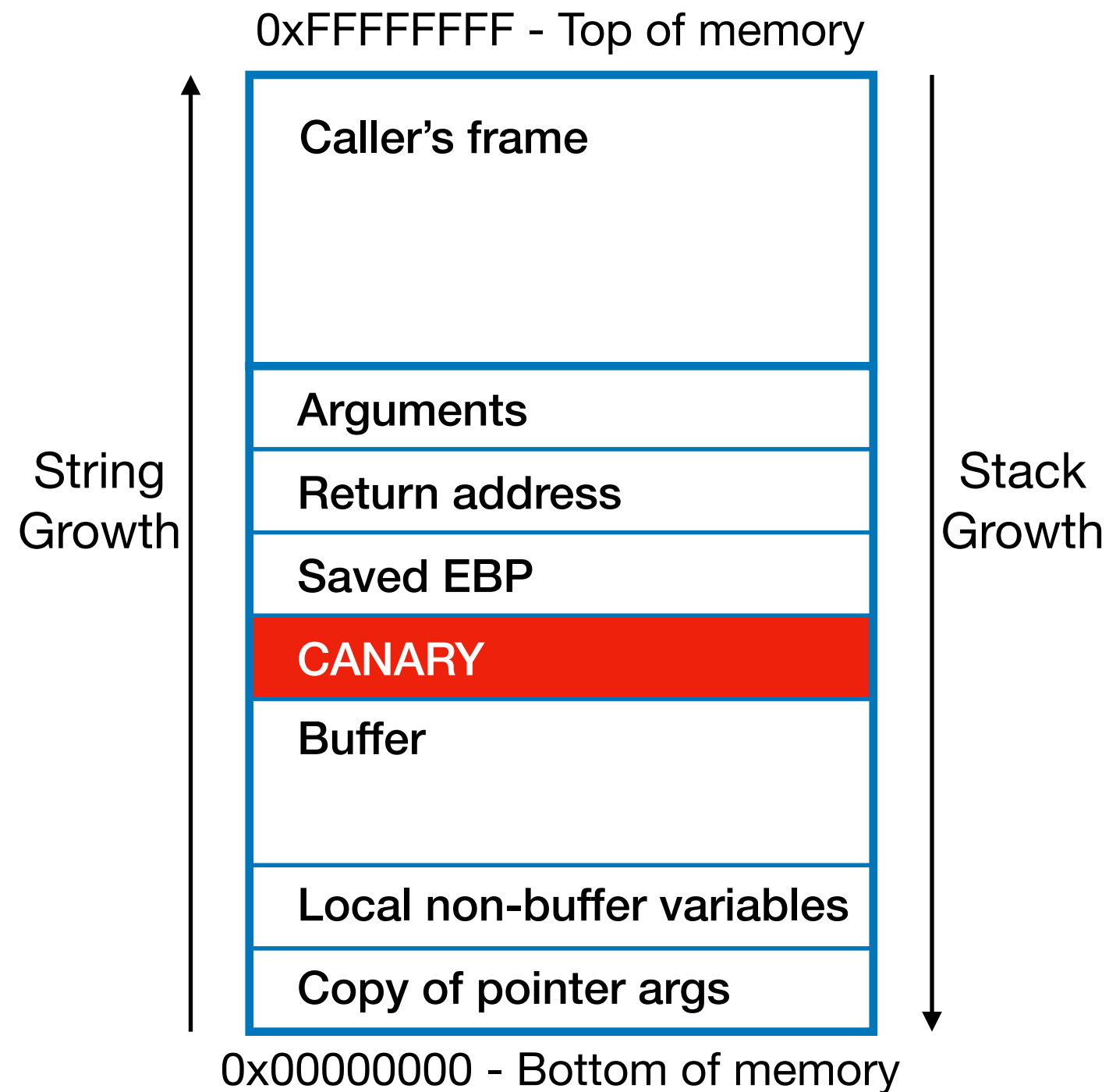
Memory Corruption Defenses (Cont.)

- Detection
 - Perform security checks during the execution of the program
 - Detect abnormal sequences of executed instructions
 - Check for integrity
- Many defense mechanisms try to prevent memory errors from being **exploitable for code injection**
 - Some defense transforms control-hijacking into denial of service

Defense #1: Canaries

- By performing run time checks for stack integrity, we can harden the executable
- **Canaries** are placed in stack frames to prevent/detect the overwrite of the return address on the stack
 - StackGuard, USENIX Security 1998
- **StackGuard** writes a canary before the return address in **each stack frame**
- When the function is returned, the canary value is verified against the stored value
- Minimal performance overhead: 8% for Apache

Stack Protection Implementations



Canary Types

- Terminator Canary
 - NULL (0x00), CR (0x0d), LF (0x0a), EOF (0xff)
 - String functions will not copy beyond a terminator
 - Attackers cannot use string functions to overwrite the return address
- Random canary
 - Random value chosen at program startup
 - Stored in location known only to the validation code and protected with unmapped pages
- XOR canary
 - Random value $\wedge$ return address

Canary Entropy

- On 32-bit x86 machines, the canary is a 32-bit value
 - 64-bit value on x86-64
- For terminator canaries, they are not completely random
 - For example, there are always 4-bit zeros
 - This makes guessing the canary much easier
 - It might be brute forced

Limitations of Canaries

- Canaries do not provide full protection
 - Canary can be learned
 - Format string vulnerabilities
 - Information leakage
 - Then it can be overwritten
 - Other pointers can still be overwritten
 - A function pointer in the function frame
 - This requires knowledge of the process memory layout
 - The **VTable** of a C++ object on the stack
 - Heap overflow

Limitations of Canaries (Cont.)

- Canaries can be bypassed by overflowing a pointer used as a destination of a strcpy() like function
- Some stack smashing techniques can overwrite the return address without touching the canaries
 - The XOR canary was introduced to protect from this attack
- Enabling canary protection requires recompile the program
 - StackGuard is implemented as a GCC patch

Defense #2: Non-Executable Memory

- In order to prevent the attack code from executing, the OS can mark the stack and the heap as **non-executable**
- Linux leverages the **NX (no-execute) bit** in the page table entry to mark certain memory areas as non-executable
 - AMD: NX bit; Intel: XD bit
 - The kernel needs to use the Physical Address Extension mode
- Windows implements the NX mechanism using **Data Execution Prevention (DEP)**
 - It can emulate the mechanism if the NX bit is not supported by the processor
- OpenBSD implements **W^X (W xor X)**
 - It forces a page to be either writable or executable but not both

Non-Executable Memory (Cont.)

- These mechanisms aim to **never** have a memory area that is both writable and executable
- Some applications may require a writable memory area that is also executable
 - Example: JIT interpreters
- Attackers can still hijack the control flow of a vulnerable application even if the memory is non-executable
 - Return into libc
 - Return oriented programming

Control Hijacking w/o Code Injection

- An attacker cannot launch code injection attacks if the memory is non executable
- The attacker can still hijack the control of a program by leveraging existing code
 - Idea: find functions or code chunks in existing libraries and glue them together to perform the desired tasks
 - Is it possible?
 - Difficult to do on your own
 - Many tools available

Defense #3: Address Space Layout Randomization

- Map the stack, the heap, the code, and the shared libraries to **random location** in process memory
 - The libraries' code need to be position-independent
 - Otherwise, run-time mapping incurs some overhead
- Jumping to a library function is **more difficult** (**not impossible**)
- Entropy
 - Windows 7: 8 bits of entropy for DLLs (1 in 256 chances)
 - 64-bit Windows 8: 24 bits of entropy
 - 32-bit Linux: 16 bits of entropy (32K attempts in average)
 - 64-bit Linux: much higher entropy

ASLR in Linux

- ASLR is enabled in Linux by default
 - `/proc/sys/kernel/randomize_va_space`
 - Can be disabled: `setarch `uname -m` -R /bin/bash`
- ASLR is implemented by the kernel in collaboration with the ELF loader
 - Stack ASLR
 - Libs/mmap ASLR
 - Exec ASLR (Requires executables in PIE format)
 - brk ASLR
 - vDSO ASLR

Exploiting ASLR

- Brute-forcing
 - Low-entropy
 - If it is possible to limit the variation space
- Attacks
 - Address leaking
 - Example #1: Format string attack
 - Example #2: ?
 - Control-flow hijacking

Defense #4: Control Flow Integrity

- Programs have a **control-flow graph (CFG)**
- Control-hijacking attacks usually result in execution paths that **do not exist** in the CFG
- Control Flow Integrity ensures that the application control flows as specified by the CFG
- Enforcing CFI:
 - Building the CFG at **compile time**
 - Instrumenting the application to check at **run time** the validity of a control transfer

Control Flow Guard (Windows 10)

- Prevent jumping to invalid locations through **indirect calls** by checking against a bit mask of all valid function entry points in the executable
- Does not prevent an attacker from jumping to a **valid wrong** function
- Hard to statically build an accurate CFG

Why does Software Have Memory Corruption Vulnerabilities?

- Programmers make mistakes
 - They are human
 - Use tools to detect/fix mistakes
- Programmers are not security-aware
 - Learn about common security flaws
- Programming languages are not well designed for security
 - Use better languages (Go, Rust, Java, Python, ...)
 - Anything but C/C++

Finding Security Problems

- Testing the security of a software is difficult
 - We need to find the abnormal/rare cases
 - Normal inputs usually would not trigger the problems
 - Security is a negative property
- How to test for security issues?
 - Random inputs
 - Or mutate the inputs with some knowledge
- How do we know we find a security problem?
 - Program crash, or any other unusual behavior
- How do we know we find all potential problems?
 - It is very difficult, but code coverage tools can help a bit

Finding Security Problems (Cont.)

- In addition to random testing (fuzzing), we can
 - Scan for known vulnerabilities
 - Ask someone to break your systems (penetration testing)
- What to do when something wrong is found?
 - Patch your software
- Patching can be difficult/expensive
 - May break some other functionalities
 - May require a stop or restart of production systems
 - It is endless

Summary

- Memory corruption may happen if an attacker can write beyond bounds of a variable
 - Attacker may be able to hijack the control of a program
- The use of safe coding, safe languages, and existing protection mechanisms can greatly reduce the risk of exploitation
- Application-logic attacks are still possible

Credits

- Some slides are based on the slides of Giovanni Vigna, Dan Boneh, Raluca Ada Popa and Nicholas Weaver

Symmetric Key Cryptography

CSCI 4130: Introduction to Cyber Security

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Cryptography

- A tremendous tool enabling **secure communication** over **insecure communication channels**
- The **basis** for many security mechanisms
 - E.g., your online browsing is secured using cryptography (foundations to TLS)
 - **Not** the solution to all security problems
 - Reliable if implemented and used properly

Three Main Goals

- Confidentiality
 - Preventing adversaries from learning the content of our private data
- Integrity
 - Preventing adversaries from altering our data
- Authenticity
 - Ensuring that the data is created by the expected user
 - Generally implies/requires integrity

Special Guests

- Alice
- Bob
- Eve
 - The attacker
 - Eavesdropper
 - Malice
- Sometimes Charlie

Communication



- **Confidentiality**: Preventing Eve from learning the message
- **Integrity**: Preventing Eve from modifying the message
- **Authenticity**: Ensuring the message is from Alice

Terminologies

- Encryption
 - Converting plaintext into ciphertext
- Decryption
 - Converting ciphertext into plaintext
- Key
 - A randomly chosen value used in encryption and decryption
- Cryptoanalysis
 - Breaking the encryption/decryption scheme by analyzing the algorithm/implementation

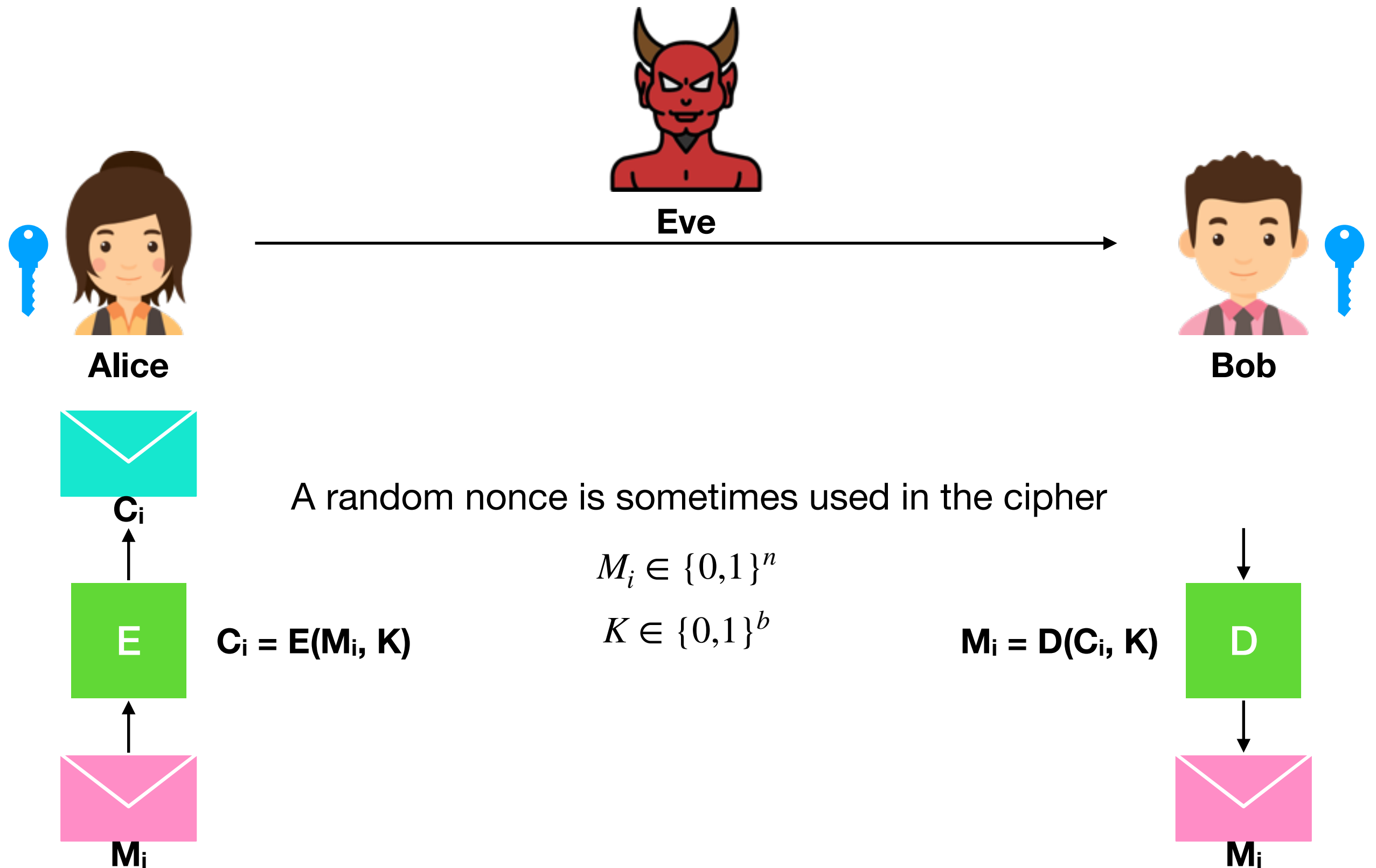
Crypto Algorithms

- Symmetric algorithms
 - Single and secret key
 - The encryption and decryption keys are the same
- Asymmetric algorithms
 - A pair of keys (One is usually public)
 - Public-key algorithms
 - The data encrypted with one key can be decrypted with the other key
 - It is computationally impractical to calculate the other key given one key

Crypto Algorithms (Cont.)

- One-way functions: It is difficult to inverse the computation
- One-way hash functions
 - Generate fixed-length output given any input
 - The output looks random
 - Collision resistance
 - Pre-image resistance
 - Second pre-image resistance
 - Can take a key as another input

Symmetric Key Encryption



Symmetric Key Encryption (Cont.)

- Our goal
 - Ensuring confidentiality
- Three algorithms
 - $\text{KeyGen}() \rightarrow K$
 - $E(M, K) = E_K(M) = C$
 - $D(C, K) = D_K(C) = M$
- Correctness
 - For any $K, M, C = E_K(M)$
 - $D_K(C) = M$

Symmetric Key Encryption (Cont.)

- Attackers's goal: **any** knowledge about M_i in addition to its length or about K
 - Slightly better than 50% probability at guessing a single bit
 - Any information about the relationship between M_i and M_j
- Assumption: K is **randomly selected and secret**
- Recognition of success: **Eve** can generally tell if he has **correctly** and **fully** recovered M_i

Eve's Attacks

- Ciphertext only
 - Eve can observe every instance of C_i
 - Eve may also have partial information about M_i
 - Example: The language is probably English
- Known plaintext
 - Eve knows part of M_i and/or entire other M_j s
 - How could this happen?
 - Example: An encrypted HTTP GET request

Eve's Attacks

- Chosen plaintext
 - Eve gets Alice to send M_j s that Eve chooses
 - How could this happen?
 - Example: Eve spoofs a message from Alice's boss and asks her to forward to Bob
- Chosen ciphertext
 - Eve gets Bob to decrypt some C_j that Eve chooses
 - How could this happen?
 - Example: Bob receives C_j and needs to decrypt it to know the message. Eve can measure the time to decrypt.
- All of the above

Security of a Cryptosystem

- Crypto algorithms can be computationally secure
 - The cost of breaking the cipher is unacceptably high
 - The time to break the cipher exceeds the useful lifetime of the encrypted information
- Cost and time are difficult to estimate
- Exhaustive key space search (brute-force)
 - The cipher may be considered secure, if trying all possible keys take the most powerful computer that humans have ever built several billion years

A Simple Cipher

- Caesar Cipher
 - Rotating each letter by K positions
 - E.g., A $\rightarrow$ C, B $\rightarrow$ D, Y $\rightarrow$ A, if $K == 2$
- A more formal definition
 - Encode the 26 alphabets as integers in $[0, 25]$
 - $E(M, K) = M + K \bmod 26$
 - $D(C, K) = C - K \bmod 26$

Attacks against Ceasar Cipher

- Brute forcing
 - How much work is required to break it?
- Frequency analysis
- Known plaintext
 - Given pairs of (M, C) to derive the key
- Chosen plaintext
 - Choses a plaintext and observe the ciphertext

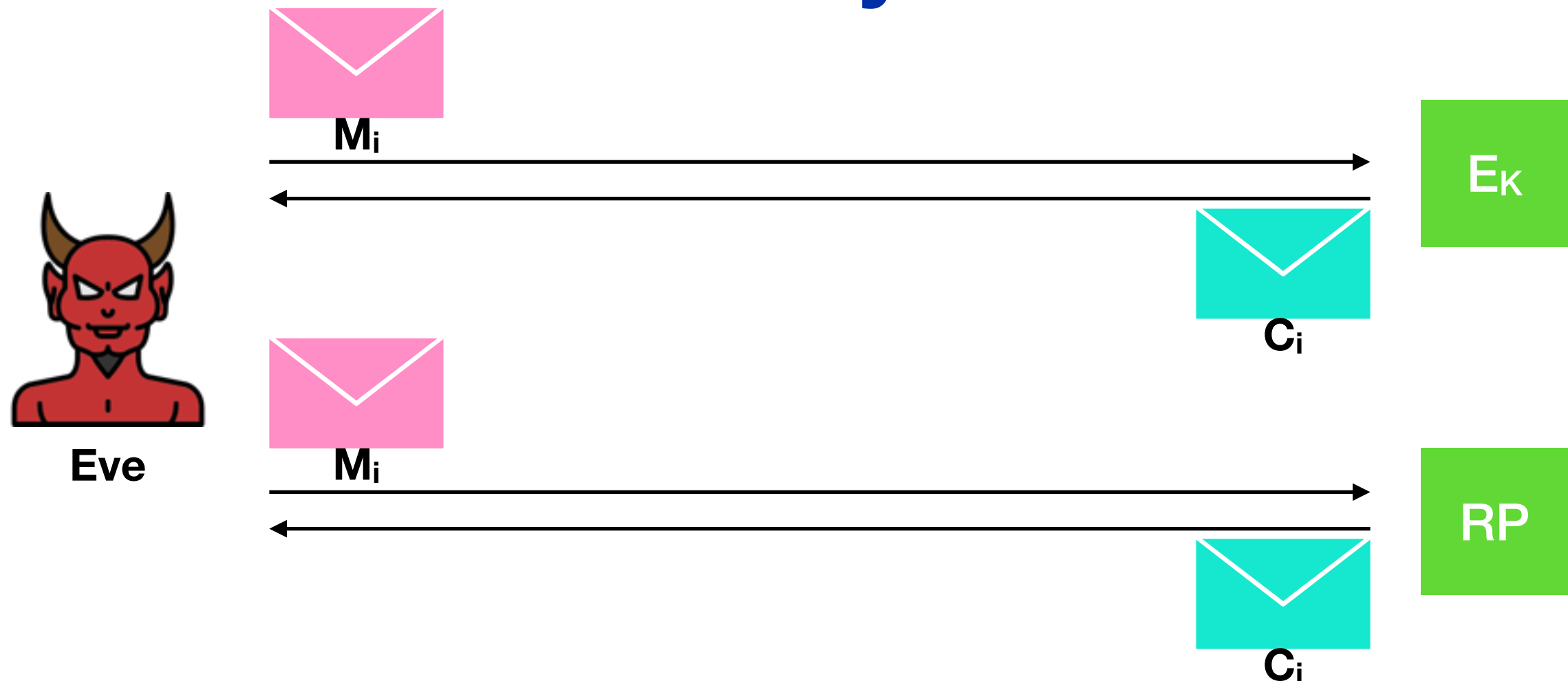
A Simple Cipher (Cont.)

- A better version
 - Rotating 1st character by K_1 , second by K_2 , ...
 - $K = \{K_1, K_2, \dots, K_n\}$
- Attacks
 - Brute forcing: How much work is required to break it?
 - Let $n = 4$
 - Frequency analysis
 - Need more ciphertext
 - Known plaintext and chosen plaintext
- A much better one?

Security of A Cryptosystem (Cont.)

- Kerckhoffs's principle
 - “A cryptosystem should be secure even if everything about the system, **except the key**, is public knowledge”
- Not enough to say that attacker cannot decrypt the ciphertext
- Attacker should not learn even **partial** information
- Attacker cannot **distinguish** which of 2 messages is encrypted in a **ciphertext**

A Security Game

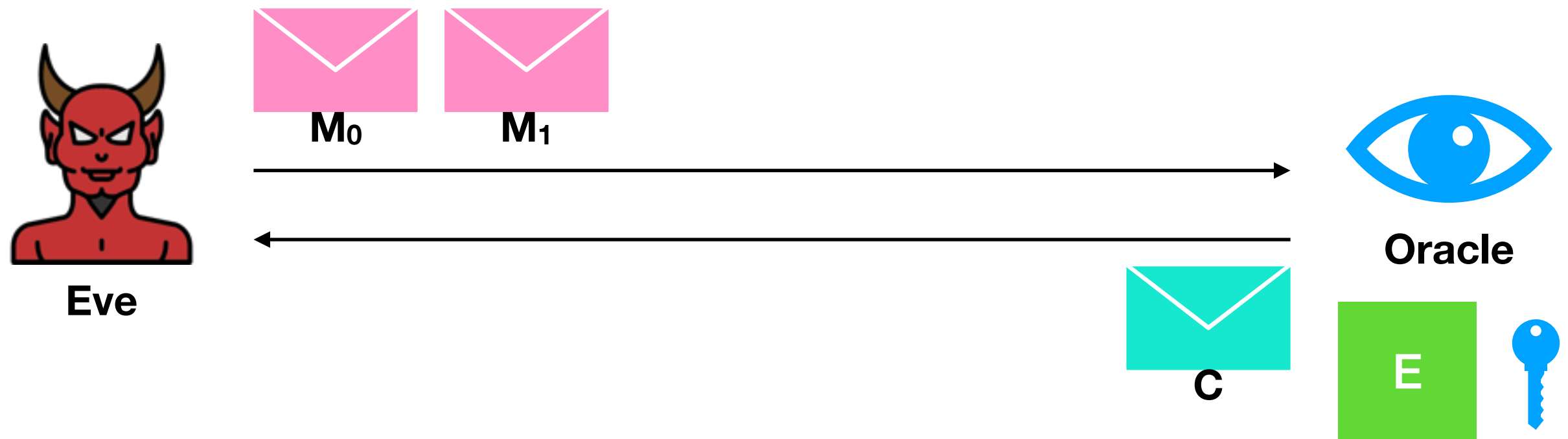


- Attacker is given two boxes, one for E_K and one for a random permutation RP
- Attacker can give input to each box and receive the output, for as many as times he/she wants
- Attacker guesses which box is E_K

Evaluating Security

- An encryption scheme meets the following condition
 - For any polynomial-time attacker, the probability to win this security game is not significantly greater than 50%
 - $\Pr[A \text{ wins}] \leq 1/2 + e$, (e.g., $e = 1/2^{128}$)
- For an unknown key K , E_K behaves like a random permutation PR
 - The attacker cannot learn the plaintext given only the ciphertext (**some confidentiality**)
- The attacker may know that $M_i == M_j$, but not the values of M_i and M_j , if $E_K(M_i) == E_K(M_j)$

Indistinguishability under Known Plaintext Attack (IND-KPA) Security Game

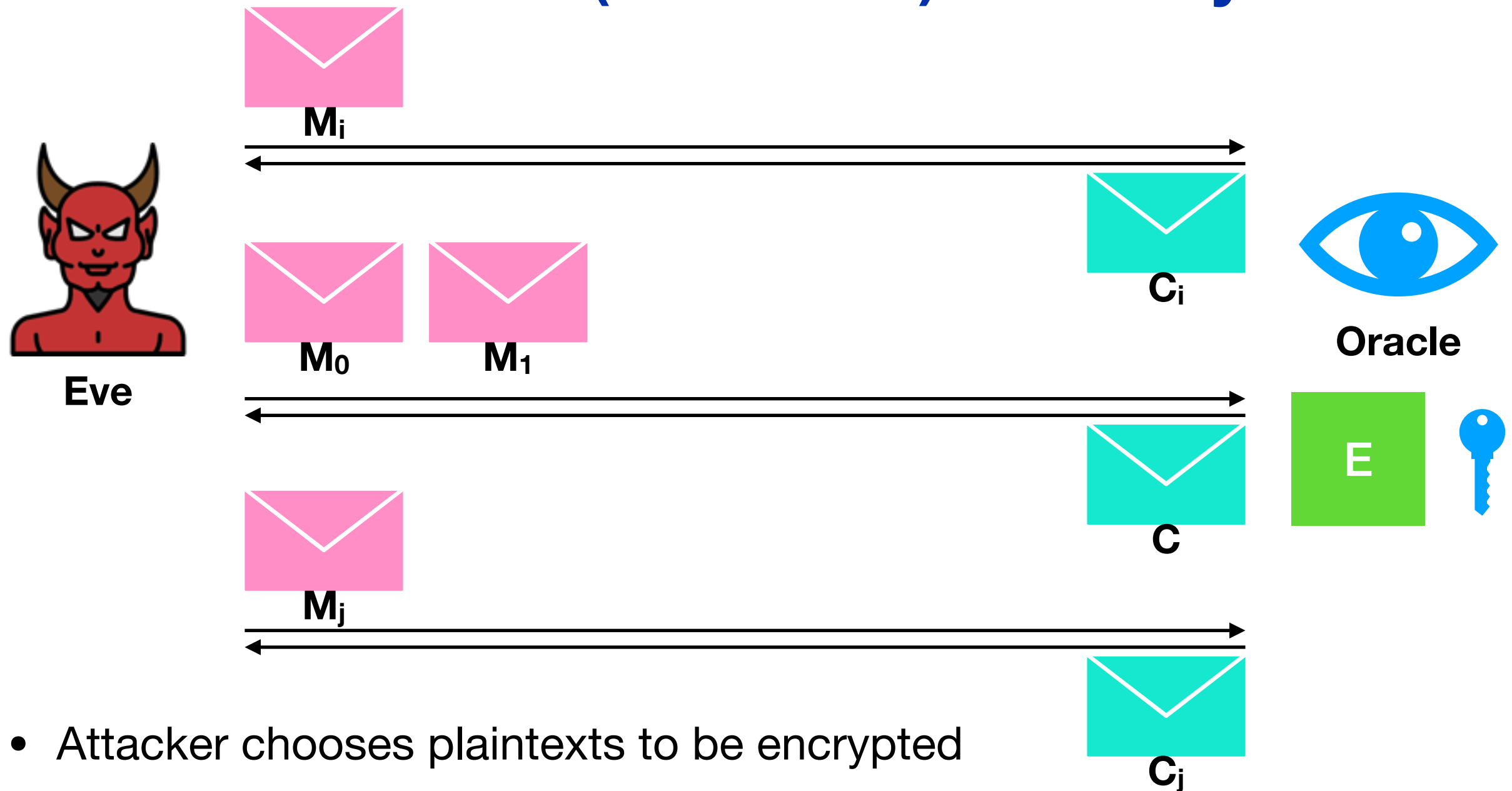


- Attacker chooses the two plaintexts
- Oracle randomly chooses a bit b and encrypts M_b
- Attacker guesses the value of b as b'
- Attacker wins iff $b == b'$

IND-KPA Security

- An encryption scheme is IND-kPA secure iff
 - For any polynomial-time attacker, the probability to win the IND-KPA security game is not significantly greater than 50%
 - $\Pr[A \text{ wins}] \leq 1/2 + \epsilon$, (e.g., $\epsilon = 1/2^{128}$)
- IND-KPA security is not a very strong level of security guarantee

Indistinguishability under Chosen Plaintext Attack (IND-CPA) Security Game



- Attacker chooses plaintexts to be encrypted
- Attacker chooses two plaintexts; Oracle randomly encrypts one
- Attacker chooses other plaintexts to be encrypted
- Attacker guesses the value of b as b' , and wins iff $b == b'$

IND-CPA Security

- An encryption scheme is IND-CPA secure iff
 - For any polynomial-time attacker, the probability to win the IND-CPA security game is not significantly greater than 50%
 - $\Pr[A \text{ wins}] \leq 1/2 + \epsilon$, (e.g., $\epsilon = 1/2^{128}$)
- IND-CPA secure is a very strong security guarantee
- Nothing leaks about the encrypted value other than its length
- IND-CPA security implies IND-KPA security

IND-CPA (Cont.)

- Are the following encryption schemes IND-CPA secure?
 - $E_K(M) = M \parallel M$
 - $E_K(M) = \text{random bits}$
 - $E_K(M) = (M+K) \bmod P$, where P is a prime number
- IND-CPA requires that the encryption scheme to be randomized
 - For the same plaintext, it outputs a different ciphertext each time

One-Time Pad

- A **random** key is chosen for **each** message M
- The key is as long as the message M

$$M_i \in \{0,1\}^n$$

$$K \in \{0,1\}^n$$

- $E(M, K) = M \wedge K$
- $D(C, K) = C \wedge K = (M \wedge K) \wedge K = M$
- The key is used only once

Security of OTP

- Shannon:
 - OTP is secure against one-time eavesdropping
 - The ciphertext reveals no information about the plaintext (except for its length)
- Claim: Given one ciphertext C , M_0 or M_1 is equally likely
 - $C = K_0 \wedge M_0$
 - $C = K_1 \wedge M_1$
 - ?

Security of OTP (Cont.)

- Is OTP IND-KPA secure?
- Proof idea:
 - Assumption: the adversary gets only one ciphertext
 - Show that $\Pr[M_0 \mid C]$ is $1/2$
- Is OTP IND-CPA secure?

One-Time Pad (Cont.)

- Problem #1: key generation
 - Need **truly** random, independent keys
- Problem #2: key distribution
 - Need a **secure** way to distribute the random key in each communication
 - If you can share the keys securely, why don't you just share the secret message?
- Problem #3: key length
 - Need the key as long as the message

Two-Time Pad

- What if we reuse a key?
 - $C_1 = E(M_1, K)$
 - $C_2 = E(M_2, K)$
- Problem?
 - $C_1 \wedge C_2 = ?$
 - $M_1 \wedge M_2$
- What does $M_1 \wedge M_2$ tells Eve?

How to Build an IND-CPA Secure Encryption Scheme?

- One-Time Pad is provably secure against one-time eavesdropping
- But it has several limitations
- Ingredients
 - One-Time Pad
 - Block Ciphers

Block Ciphers

- Building block:
 - A function $E: \{0, 1\}^n \times \{0, 1\}^k \rightarrow \{0, 1\}^n$
 - Fixing the key K : $E_K: \{0, 1\}^n \rightarrow \{0, 1\}^n$
 - $E_K(M) = E(M, K)$
- Properties
 - **Correctness**: E_K is a permutation (bijective)
 - **Efficiency**: Computation is very fast
 - **Security**: The permutation looks random without knowing K

Block Ciphers (Cont.)

- Data Encryption Standard (DES)
 - Designed in late 1970s
 - 56-bit key, 64-bit block
 - Tweaked by NSA (64-bit key -> 56-bit key)
 - Unbroken for decades
- Advanced Encryption Standard (AES)
 - 128, 196, or 256-bit key
 - 128-bit block
 - Not proven secure
 - No known flaw

Block Cipher Security

- For an unknown key K , E_K behaves like a random permutation PR
 - The attacker cannot learn the plaintext given only the ciphertext
- Block ciphers provide *some confidentiality*
- With the same key, the block cipher produces the *same* output on *same* inputs
- Are block ciphers IND-CPA secure?
 - No, because they are deterministic
 - How to win the IND-CPA game?

Using Block Ciphers

- We want two properties
 - Encrypt messages of arbitrary lengths
 - IND-CPA security even when reusing the same key to encrypt many messages
- Like OTP, block ciphers work on fixed-length input
 - M is shorter: padding
 - M is longer: applying the block cipher repeatedly
 - Called a **block cipher mode**
 - Can hide that same block is encrypted twice

Modes of Operation

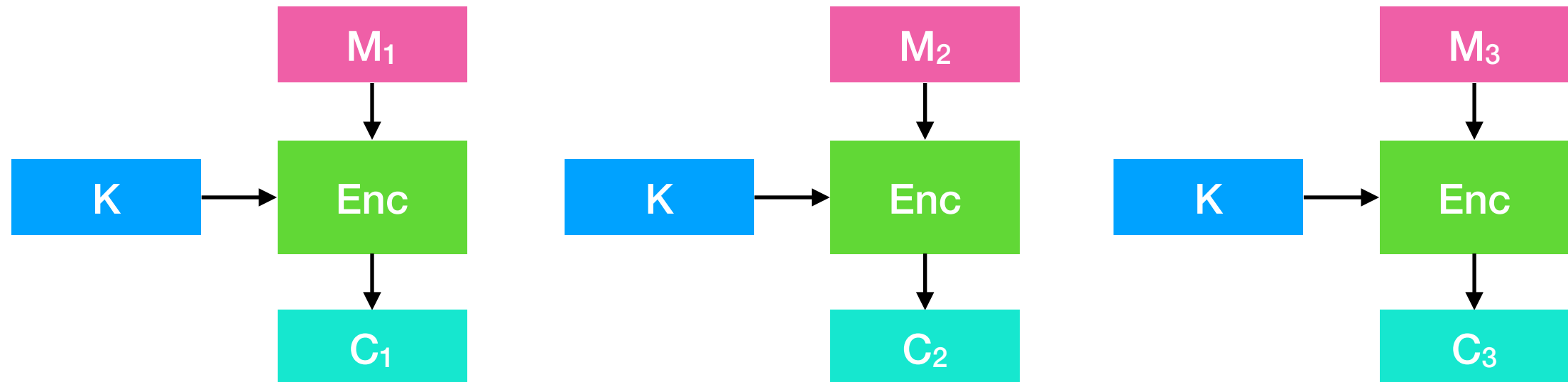
- Chain block ciphers in certain modes of operation
 - Invoke block cipher multiple times on inputs related to other blocks
- Need some **initial randomness**
 - IV (Initialization Vector)
- Why?
 - To prevent the encryption scheme from being deterministic

Electronic Code Book (ECB) Mode

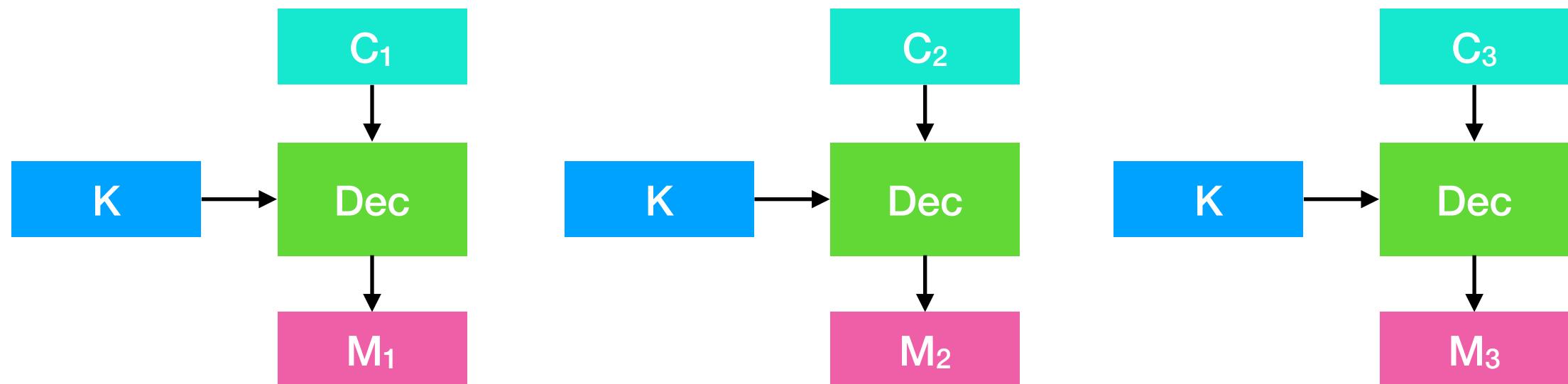
- Simplest mode
- Split a long message into n-bit blocks $M_1, M_2, \dots$
- Each block is encrypted/decrypted **independently**, using the **same key**
 - $C_i = E_k(M_i)$
 - $M_i = D_k(C_i)$
- Concatenate the encrypted/decrypted blocks to produce the output

ECB Mode (Cont.)

Encryption



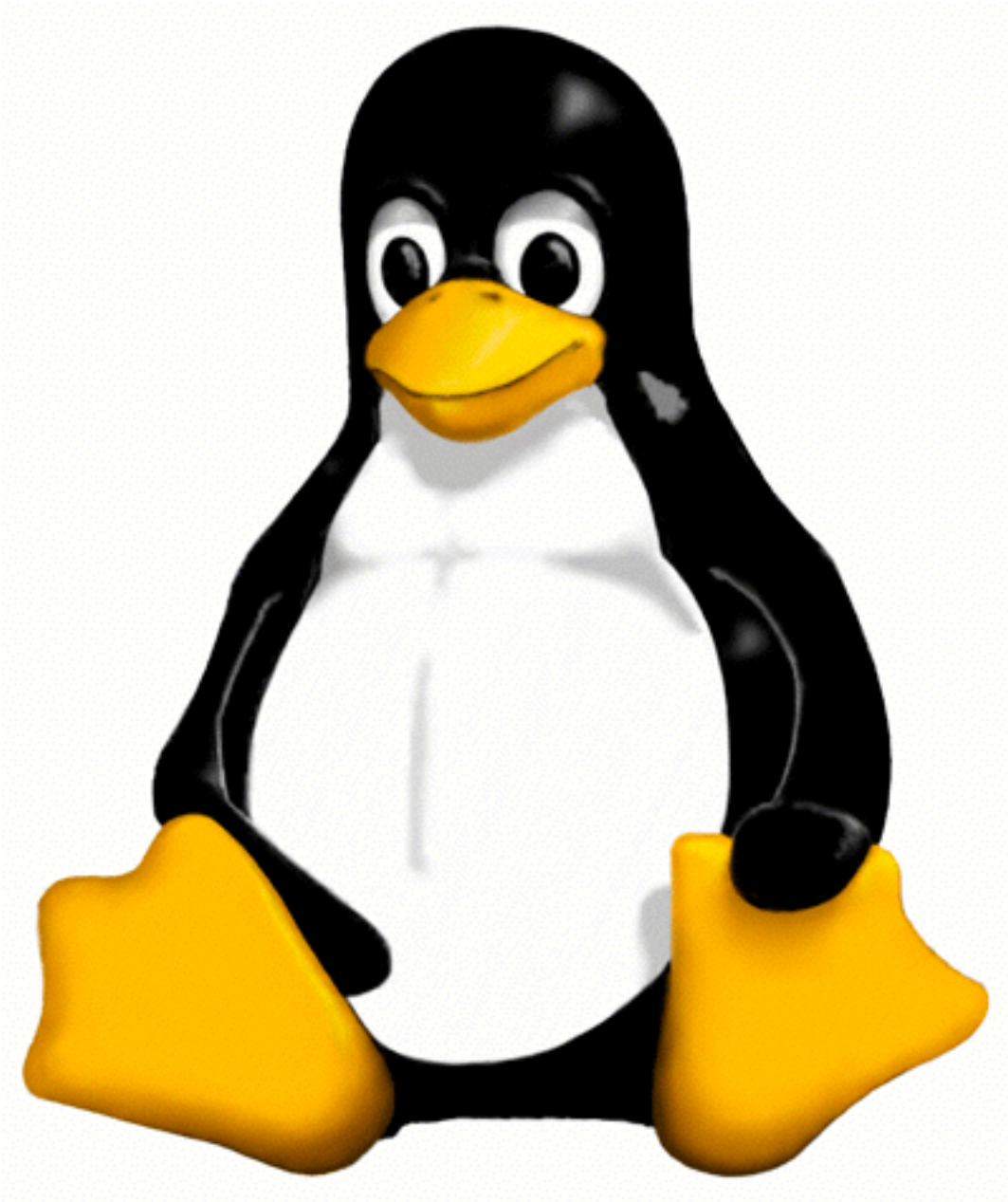
Decryption



Security of ECB Mode

- Is there any problem with ECB?
- Is ECB mode IND-CPA secure?

Security of ECB Mode (Cont.)

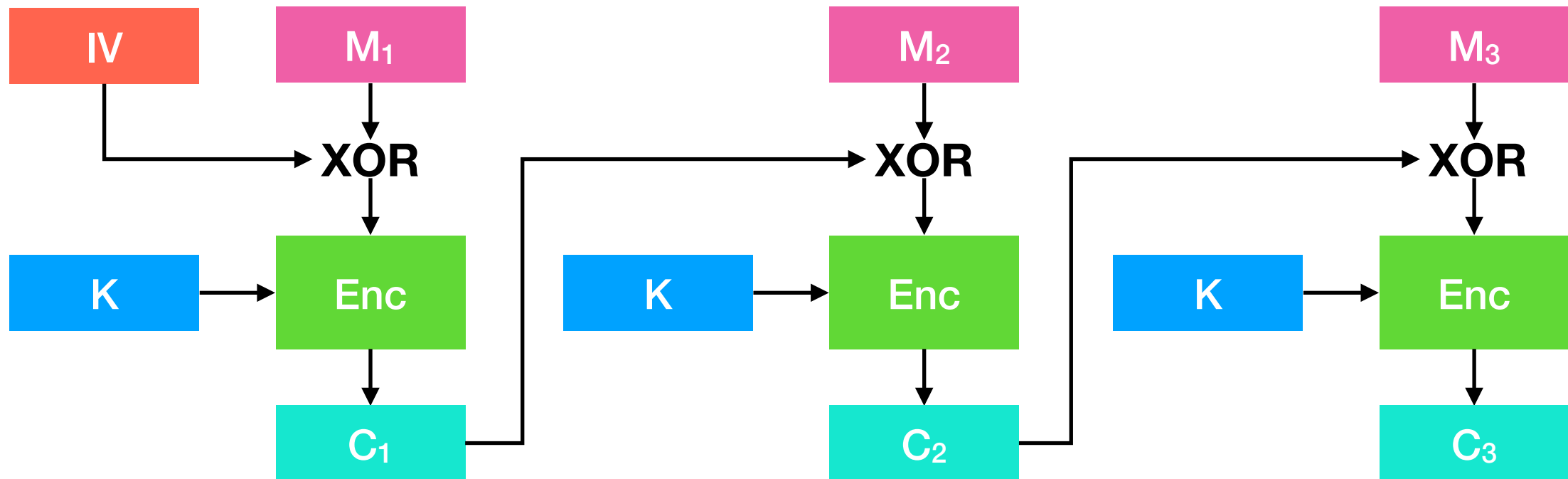


Cipher Block Chaining (CBC) Mode

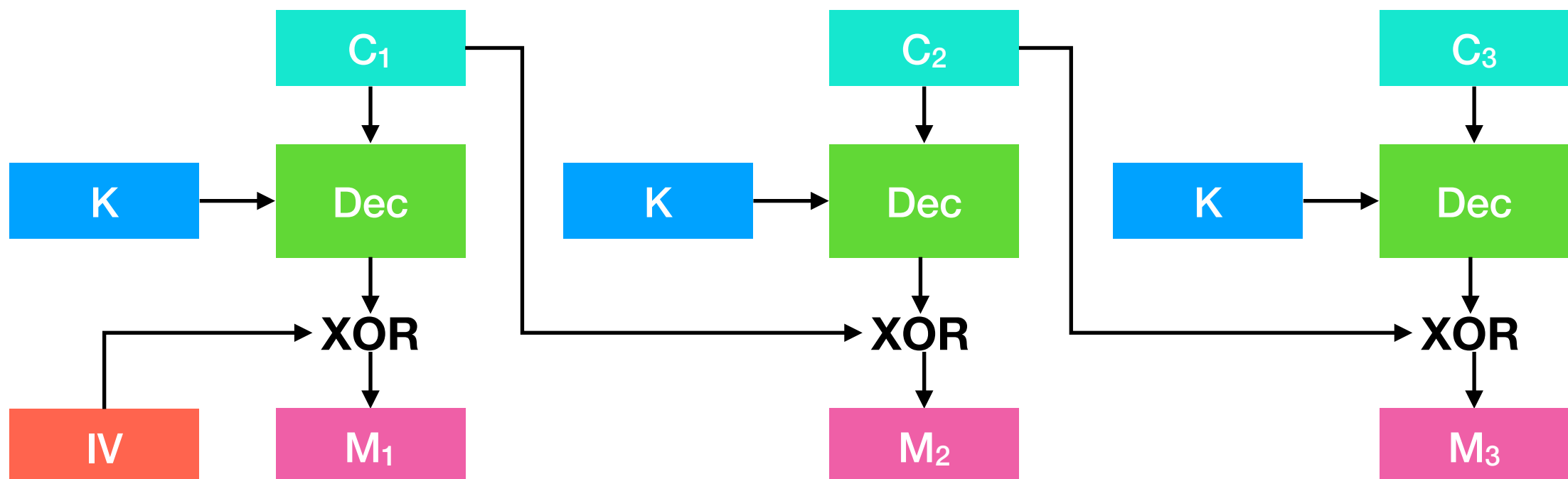
- A better mode: Including a varying, known nonce
- Split a long message into b -bit blocks $M_1, M_2, \dots$
- Choose a one-time random IV (Initialization Vector)
- Chain the block ciphers by including the output of the previous block as an input of the current block
- Final output is IV + concatenation of output blocks

CBC Mode (Cont.)

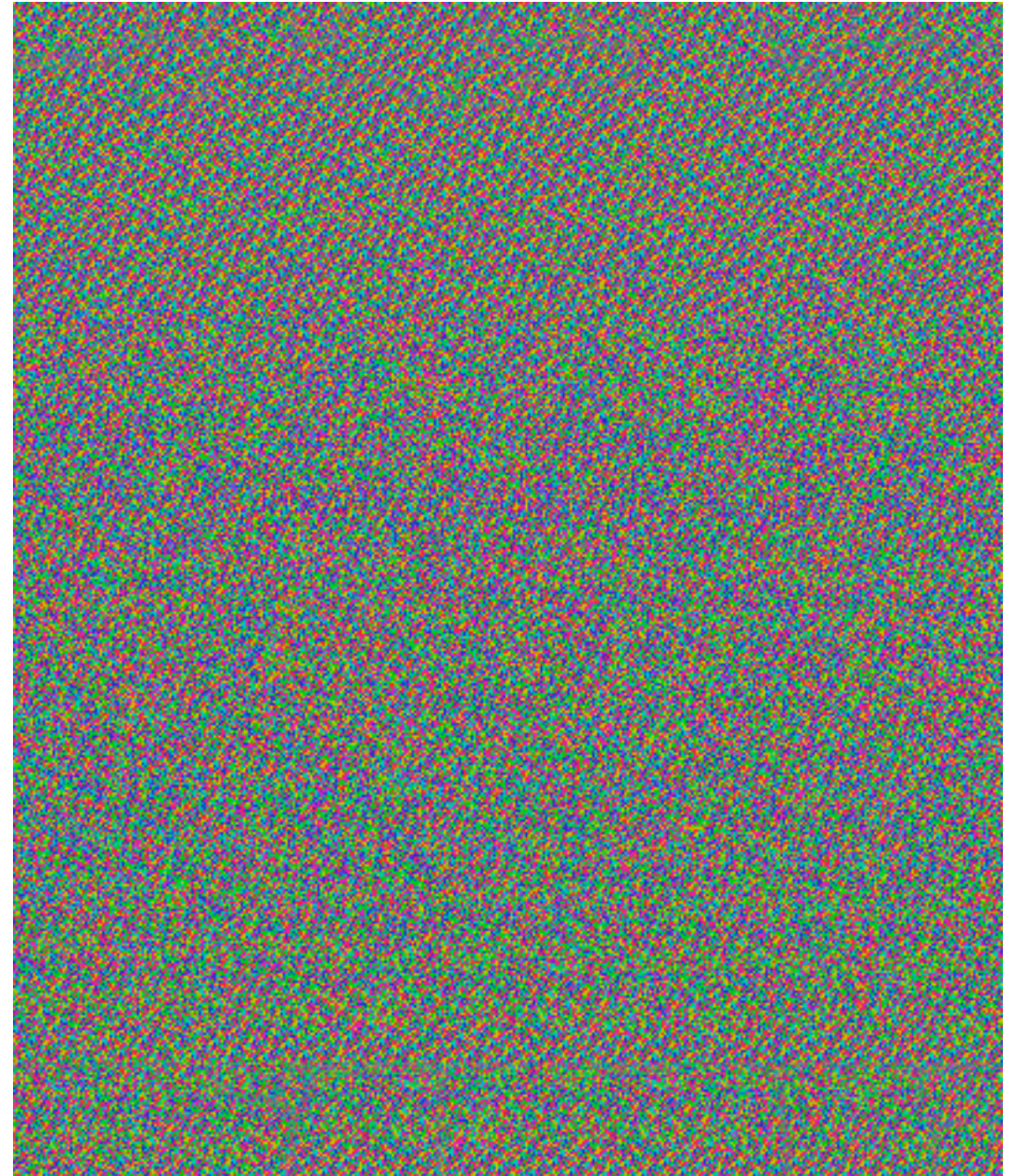
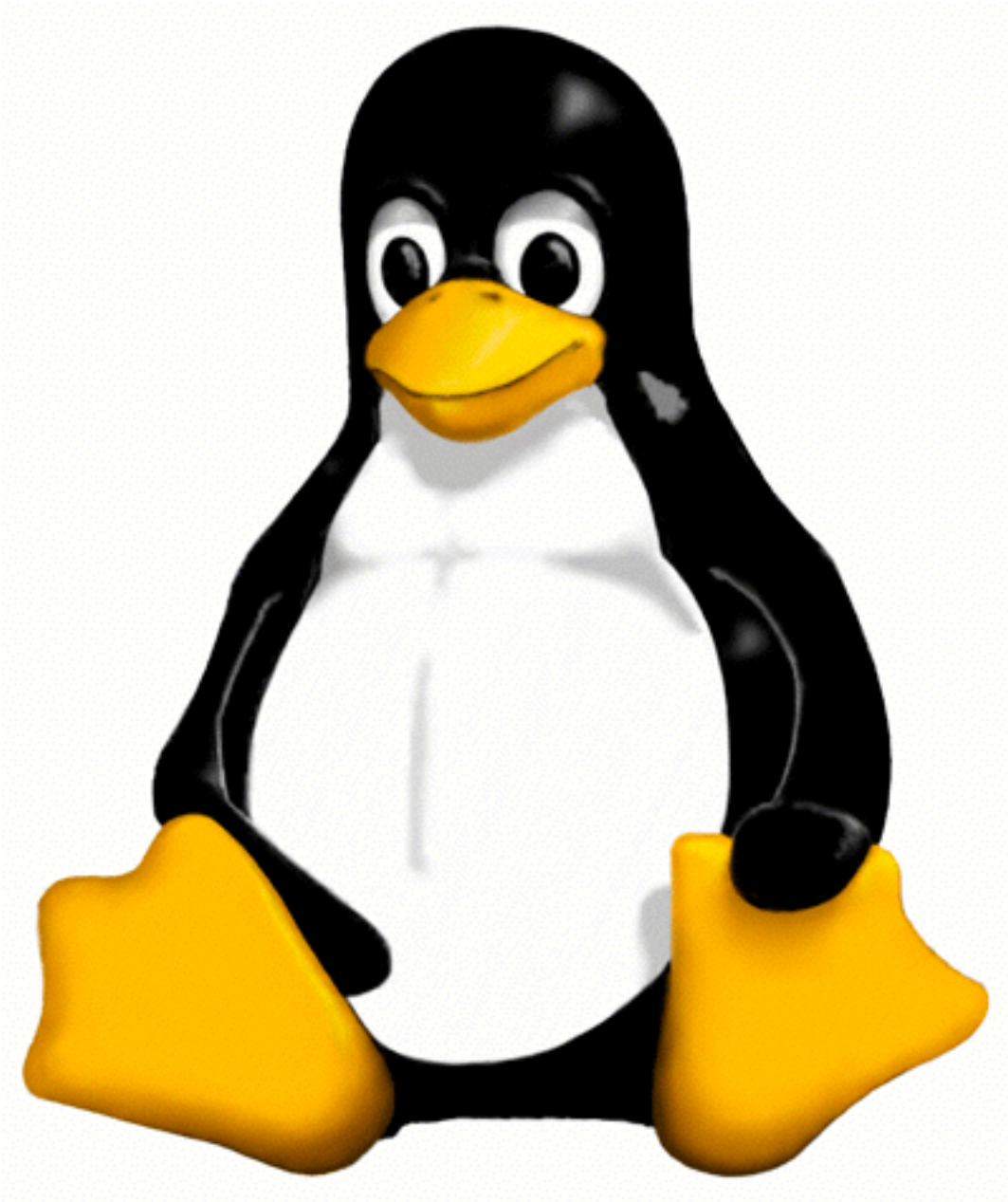
Encryption



Decryption



CBC Mode (Cont.)



CBC Mode (Cont.)

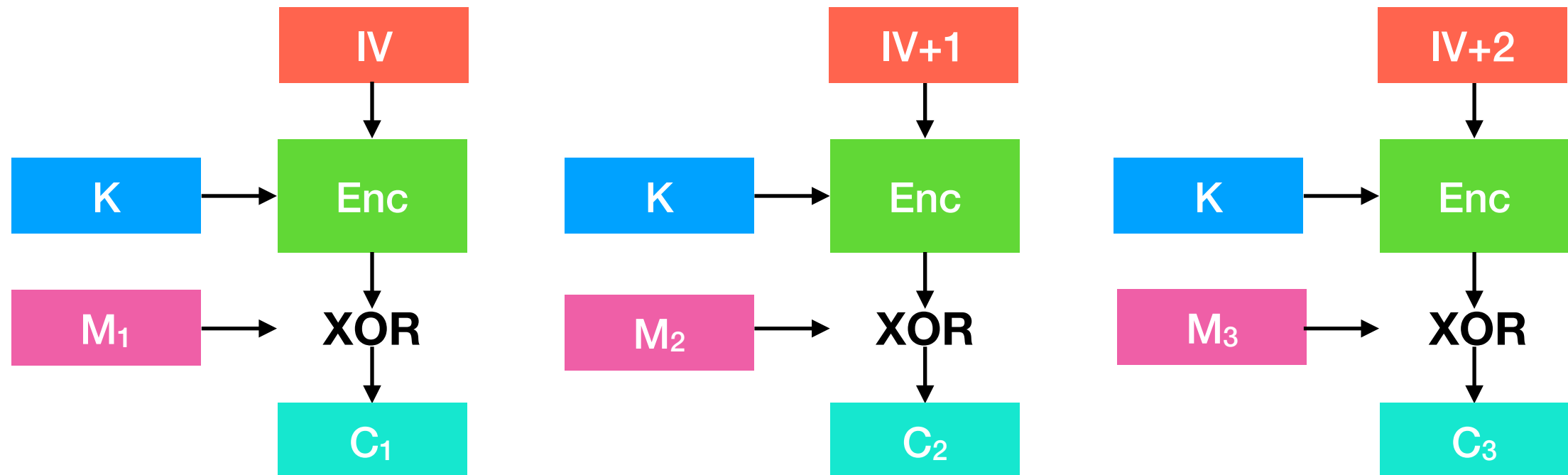
- Achieves IND-CPA security
- CBC mode is popular, still widely used
- Disadvantages
 - Sequential encryption
 - Hard to parallelize

Counter (CTR) Mode

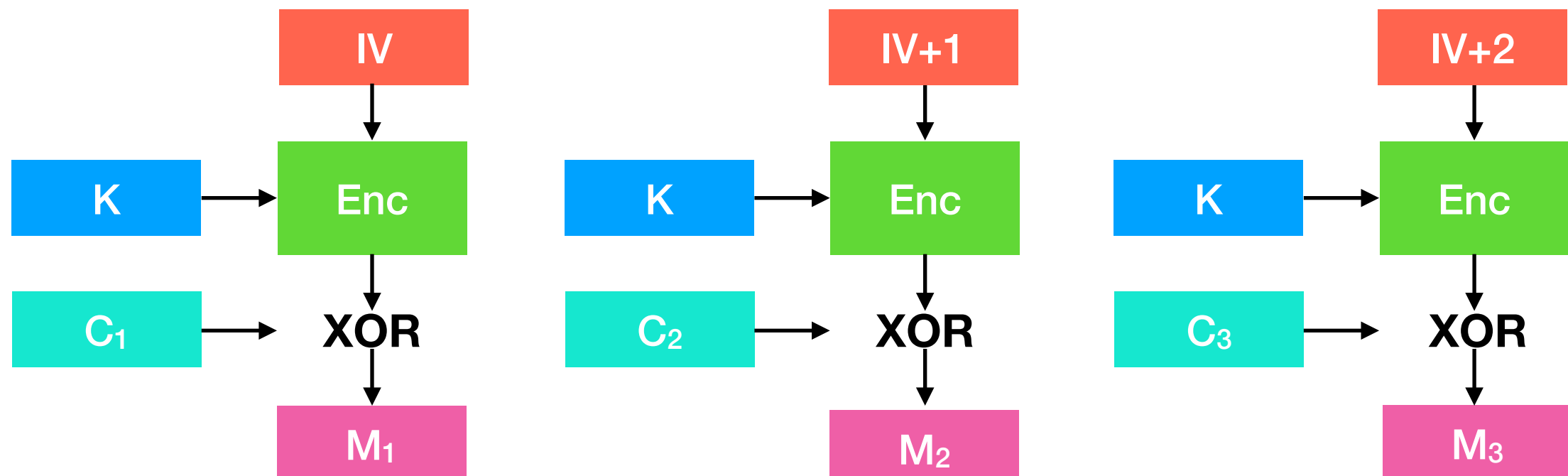
- CBC mode is widely used but hard to parallelize
- CTR mode
 - Parallelize encryption/decryption
 - Also provably secure
 - Select a random IV/nonce
 - Increment the IV for each block
- Note: CTR decryption uses block cipher's encryption, not decryption

CTR Mode (Cont.)

Encryption



Decryption



CBC vs. CTR

- Security
 - Both are IND-CPA, if no reuse of IV
- Performance
 - Both modes require the same amount of computation
 - CBC is parallelizable for only decryption
 - CTR is parallelizable for both encryption and decryption

Pseudorandom Number Generators

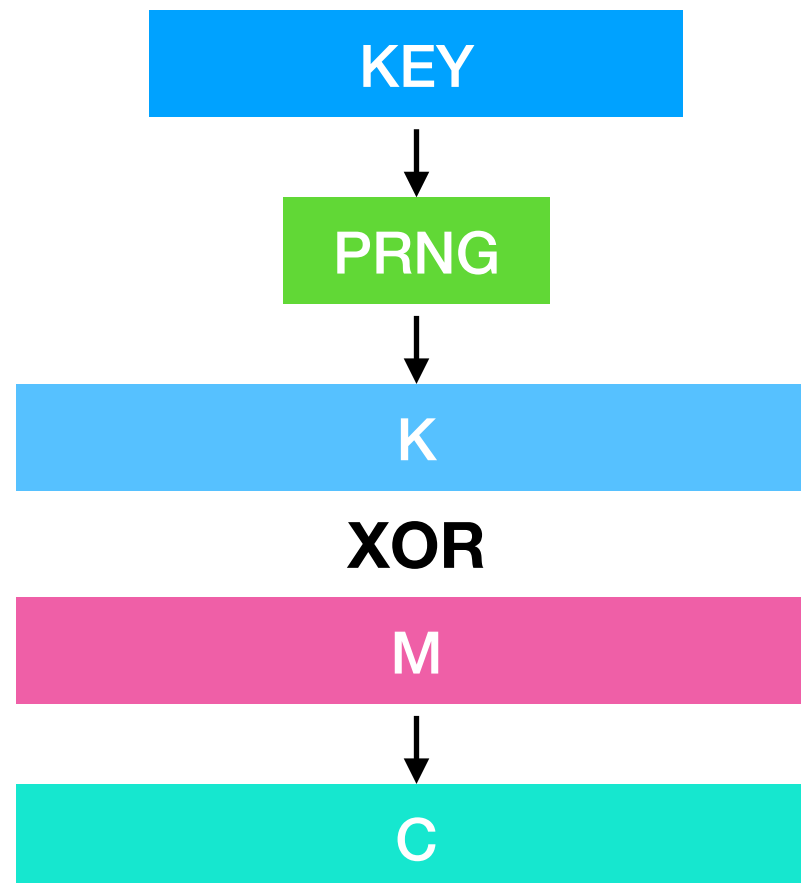
- Given a seed, outputs sequence of seemingly random numbers
- Can output arbitrarily many random bits
- Keep internal state
- Can a PRNG be truly random?
 - No
 - Why?

Pseudorandom Number Generators (Cont.)

- A cryptographically strong PRNG “looks” truly random to an attacker
 - Attacker cannot distinguish its output from a truly random sequence
- Generating randomness
 - `/dev/random`
 - Hardware interrupts
 - Hardware RNG: Intel RdRand instruction
- How to build a cryptographically strong PRNG?

Stream Ciphers

- Block cipher: fixed-size, stateless, requires modes to securely process longer messages
- Stream cipher:
 - Keep states from previous messages
 - Generate a one-time pseudorandom key K



Stream Ciphers (Cont.)

- Encryption
 - Choose a random IV
 - $E(M, K) = \text{PRNG}(K, \text{IV}) \text{ XOR } M$
- Decryption
 - $D(C, K) = \text{PRNG}(K, \text{IV}) \text{ XOR } C$
- Similar in spirit to OTP
- Can it encrypt message of any length?
 - PRNG can produce any number of random bits
- Can also encrypt a message multiple times

Symmetric Encryption Summary

- Desirable security is IND-CPA
- Block ciphers can be used to build IND-CPA secure encryption schemes with certain modes of operation
- Stream ciphers provide another way to encrypt
- Confidentiality is insufficient for most applications
- We need also to defend against active (tampering) attacks

Integrity and Authentication

- Why do we need integrity and authenticity?
 - Active attacker
 - Can inject new messages (ciphertext)
 - Can replay previous cipher texts
 - Can cause messages to be reordered or discarded
 - Man-in-the-middle attacker
 - Can be much more powerful than just eavesdropping

Integrity and Authentication (Cont.)

- Goals:
 - Given a message, prove it is not altered
 - Given a message, prove it is indeed sent from the sender
- Approach: Providing integrity and authentication with cryptographically strong hash functions
- Examples:
 - MD5: 128 bits, broken
 - SHA-1: 160 bits, broken
 - SHA-256: 256 bits, not broken yet

Cryptographically Strong Hash Functions

- Properties required
 - One-way
 - Given y , difficult to find x s.t. $\text{Hash}(x) = y$
 - Collision resistance
 - Difficult to find x and x' s.t. $\text{Hash}(x) = \text{Hash}(x')$
 - Second pre-image resistance
 - Given x , difficult to find x' s.t. $\text{Hash}(x) = \text{Hash}(x')$

SHA-256

- SHA-256/SHA-384 are two parameters for the SHA-2 hash algorithm, returning 256-bit or 384-bit hashes
 - Can work on inputs of arbitrary length
- Is vulnerable to a length-extension attack
 - S is secret
 - Attacker knows $\text{len}(S)$, $H(S)$
 - Attacker can calculate $H(S||M)$ for an M
 - Works because all the internal state at the point calculating $H(S||\dots)$ can be derived from $H(S)$ and $\text{len}(S)$
- SHA-3 standard does not have this property

Message Authentication Code

- Cryptographically strong hash functions are not sufficient for integrity and authentication
 - Attackers can also compute a valid hash value
- Message Authentication Code (MAC) provides message integrity and authentication
 - It does not provide confidentiality
 - Needs a secret shared key K
 - The message itself can be in plaintext
- Idea: Compute and verify a tag of the message that only Alice and Bob can produce

MAC (Cont.)

- Alice
 - Computes $T = \text{MAC}(M, K)$
 - Sends $\{M, T\}$
- Bob
 - Receives $\{M', T'\}$
 - Computes $T'' = \text{MAC}(M', K)$ and compare T' to T''
- Integrity is provided if the attacker cannot
 - Find M^* and T^* s.t. $T^* = \text{MAC}(M^*, K)$
 - Find M^* s.t. $T = \text{MAC}(M^*, K)$
- Cryptographically strong hash functions + secret shared key

Hash-MAC (HMAC)

- Most widely used MAC on the Internet
- Building secure MAC
 - H is a cryptographically strong hash function
 - Pad_i , Pad_o are publicly known strings
 - K is a secret key
- $\text{HMAC}(M, K) = H(K^{\wedge}\text{Pad}_o \parallel H(K^{\wedge}\text{Pad}_i \parallel M))$
- Believed to be secure even if H is not cryptographically strong

Using MAC

- MAC in general do not provide any guarantee on confidentiality of the message
- It is recommended to compute MAC
 - Using a separate key
 - On the ciphertext
- K_E : Encryption key; K_I : MAC Key
- Option 1 (SSL): $C = \text{Enc}(M \parallel \text{MAC}(M, K_I), K_E)$
- Option 2 (SSH): $C = \text{Enc}(M, K_E) \parallel \text{MAC}(M, K_I)$
- Option 3 (IPsec): $C = \text{Enc}(M, K_E) \parallel \text{MAC}(\text{Enc}(M, K_E), K_I)$

Symmetric Cryptography Summary

- Pros:
 - Confidentiality: Strong
 - Integrity: Weak
 - Authenticity: Weak
- Cons:
 - Secret key sharing is difficult, typically requiring another secure communication channel
 - Need more keys if more parties are involved
 - Cannot prove that the message is sent/encrypted by one of the involved parties

Credits

- Some slides are based on the slides of Vern Paxson, Dan Boneh, Giovanni Vigna, Raluca Ada Popa and Nicholas Weaver

Asymmetric Key Cryptography

CSCI 4130: Introduction to Cyber Security

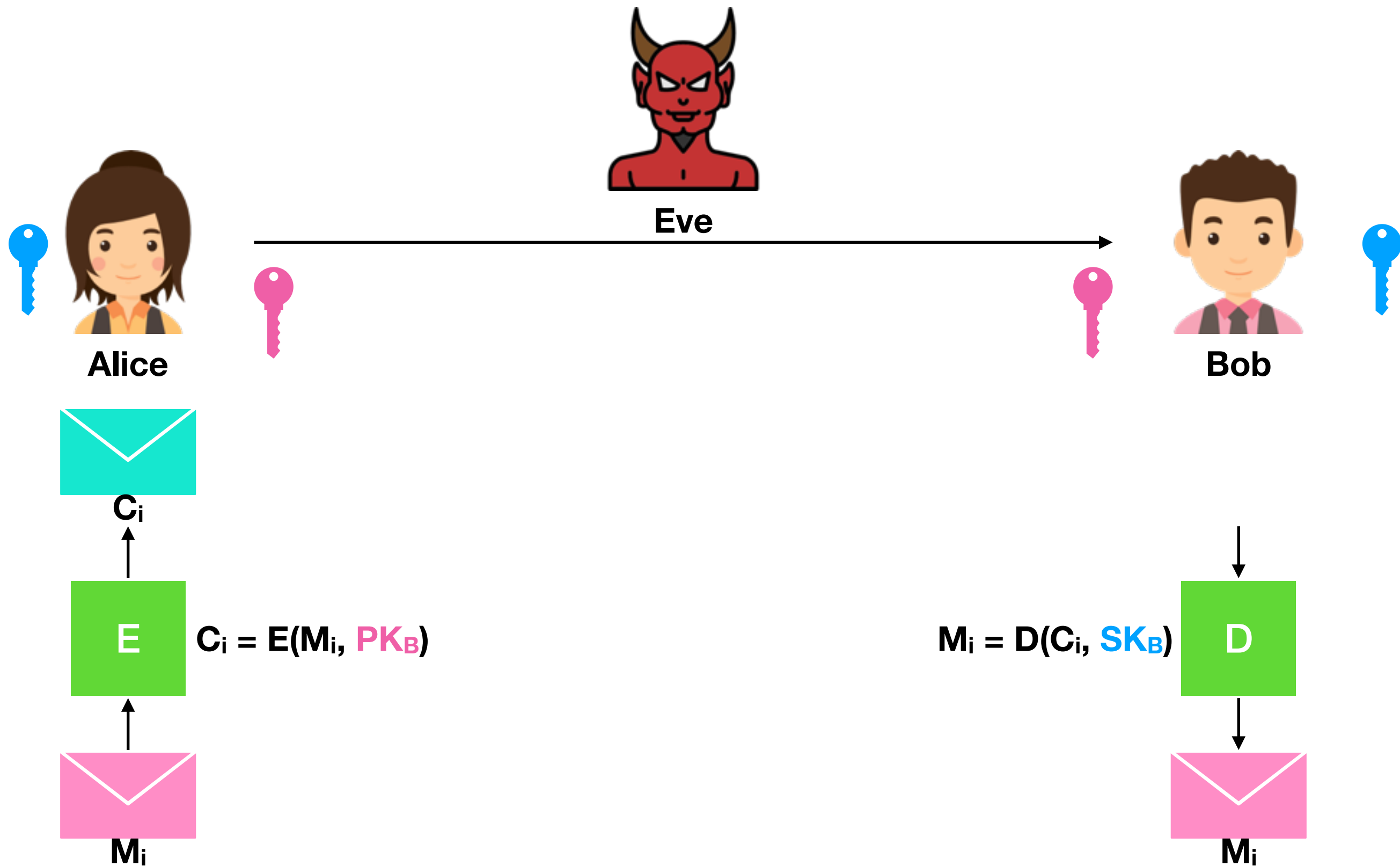
Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Asymmetric Cryptography

- Solve the problem of secret key management/distribution in symmetric encryption
- Each party has a pair of keys {PK, SK}
- The public keys are known
- Asymmetric key encryption
 - Alice encrypts the message with Bob's public key
 - Bob decrypts the message with his private key

Asymmetric Key Encryption



Building Blocks of Asymmetric Key Encryption

- Key Generation Algorithm
 - The generated public key (PK) cannot be used to easily compute the secret key (SK), or vice versa
- Trapdoor permutations
 - One-way permutation without special information
- Encryption Algorithm
 - $C = F(PK, M)$
 - Difficult to inverse without knowing the secret key
- Decryption Algorithm
 - $F^{-1}(SK, C) = M$

One-Way Functions

- A function f such that
 - Given x , it is easy to compute $f(x)$
 - Given y , it is difficult to find any x , s.t., $f(x) = y$
 - The difficulty is measured in a polynomial time computer
- Examples
 - $f(x) = 1$
 - $f(x) = x+1$
 - $f(x) = E_K(x)$, where E_K is a block cipher with a random key K

Euler's Totient Function

- Totient of n
 - $\phi(n)$
 - number of positive integers that are relatively prime to n
 - # of k , $0 < k < n$, s.t. $\gcd(k, n) = 1$
- $\phi(p) = p-1$, if p is a prime number
- $\phi(pq) = \phi(p)\phi(q)$, if p, q are relatively prime
 - The function is multiplicative
- Euler's theorem
 - If a and n co-prime, then $a^{\phi(n)} = 1 \pmod n$

Finding Big Primes Quickly

- 1. Generate a large random number p
- 2. Generate a random integer a : $1 < a < p-1$
- 3. Test $a^{p-1} \bmod p = 1$
 - If not, go to 1, find another p
 - Otherwise, go to 2, generate another a , until we have made enough iterations
 - There exist non-prime numbers for which the equation in Euler's theorem holds
 - They are very rare

RSA Key Generation

- Find **large** primes p, q
- Compute $n = p \cdot q$
- Compute $f(n) = (p-1)(q-1)$
- $f(n)$ is difficult to compute even knowing n
- Choose an e that co-primes with $f(n)$ s.t. $2 < e < f(n)$
- Compute $d = e^{-1} \bmod f(n)$, which is the multiplicative inverse of e , modulo $f(n)$
- PK: $\{n, e\}$
- SK: $\{d\}$

RSA Encryption

- M is an unsigned integer that is smaller than n
- Encryption
 - $C = E(M, PK) = E_{\{n, e\}}(M) = M^e \bmod n$
- Decryption
 - $D(C, SK) = D_{\{d\}}(C) = C^d \bmod n$
 - $= M^{ed} \bmod n$
 - $= (M^{ed-1})M \bmod n$

RSA Encryption (Cont.)

- $e \cdot d = 1 \bmod f(n) \Rightarrow e \cdot d - 1 = k \cdot f(n)$ for some k
 - $(M^{ed-1})M \bmod n = (M^{kf(n)})M \bmod n$
 - $= ((M^{f(n)})^k)M \bmod n$
 - $= (1^k)M \bmod n$ (Euler's Theorem)
 - $= M \bmod n = M$

RSA Encryption (Cont.)

- What if M is no less than n ?
 - Use public-key encryption to exchange a symmetric encryption key
 - Encrypt the message with symmetric encryption
 - This is how public-key encryption commonly used in practice

Diffie-Hellman Key Exchange

- Alice and Bob agree to use a prime p , and a base g
 - g is a primitive root modulo p
 - For every a coprime to p , there is a k , s.t. $g^k = a \pmod{p}$
 - g is a generator of the multiplicative group of integers module p
- Alice chooses a secret a
 - Alice then sends Bob $A = g^a \pmod{p}$
- Bob chooses a secret b
 - Bob then sends Alice $B = g^b \pmod{p}$

Diffie-Hellman Key Exchange (Cont.)

- Alice computes $s = B^a \bmod p$
- Bob computes $s = A^b \bmod p$
- The computation of modular exponentiation (i.e., $g^a \bmod p$) can be done efficiently even for large numbers
- Discrete Logarithm Problem
 - Given only g , p and $A = g^a \bmod p$, it is very difficult to find a , if p is very large (e.g., 600 digits)
- Then, an attacker cannot learn the secret even with the fastest computer

Ephemeral Diffie Hellman

- $s = g^{ab} \bmod p$ is used as the basis for a session key
 - A symmetric key used to protect the communication between Alice and Bob
 - If either a or b is random, s is random
- When Alice and Bob stop the communication, the secrets are discarded
 - This provides forward secrecy
 - Alice and Bob don't retain any information that a later attacker who compromise Alice or Bob's secrets could use to decrypt the messages exchanged with s

MITM Attacks against DH

- The attacker is an active MITM attacker
 - He/She can intercept the message exchanged in the DH protocol
- Eve generates his/her own a' , b'
- Instead of sending $A = g^a \bmod p$, and $B = g^b \bmod p$
- Eve sends $A' = g^{a'} \bmod p$, and $B' = g^{b'} \bmod p$
- Alice will compute $sA' = g^{ab'} \bmod p$
- Bob will compute $sB' = g^{a'b} \bmod p$
- Eve also knows these values!

Digital Signatures

- Goal: Binding a document to the author
 - Attackers can copy Alice's signature from one document to another document
- Solution: Making the signature depend on the document
 - MAC?
 - The signature should be verifiable (publicly)
- $S = \text{sign}(M, SK) = F^{-1}(H(M), SK)$
- $\text{verify}(M, S, PK) = \text{accept}$ if $F(S, PK) = H(M)$

RSA Digital Signatures

- Alice generates a public/private key pair $\{n, e\}$ and $\{d\}$
 - Not recommended to be used for encryption
- Signing a message M
 - $S = \text{sign}_{\{d\}}(M) = \text{RSA}_{\{d\}}(H(M)) = H(M)^d \bmod n$
- Validating the signature
 - $\text{verify}_{\{n, e\}}(M, S) = \text{true}$ iff $H(M) = S^e \bmod n$
- It provides non-repudiation if Alice's public signature key is well-known, because anyone can verify the signature

Asymmetric Key Encryption

- Pros:
 - Key distribution
 - Confidentiality
 - Integrity (weak): Message tampering is detected when decrypting
- Cons:
 - No authentication
 - Repudiation

Asymmetric Key Encryption Summary

- Public Key
 - Something that everyone knows
- Private Key
 - A secret belonging to a specific entity
- Diffie-Hallman
 - A key exchange protocol to share secret
- RSA
 - Provide a ciphertext using the recipient's public key
- RSA Signatures
 - Provide a message that anyone can prove was generated with a private key

A Better Asymmetric Key Encryption

- A better asymmetric key encryption
 - Alice encrypts the message with her private key
 - Alice then encrypts the message with Bob's public key
 - Bob decrypts the message with his private key
 - Bob then decrypts the message with Alice's public key

A Better Asymmetric Key Encryption (Cont.)

- Pros:
 - Confidentiality
 - Integrity
 - Authentication
 - Non-repudiation
- Cons:
 - Vulnerable to MITM attacks

MITM Attacks

- Alice sends her public key to Bob
- Eve intercepts the message and sends Eve's public key to Bob
- Bob sends his public key to Alice
- Eve intercepts the message and sends Eve's public key to Alice
- Alice sends a message encrypted with Eve's public key
- Eve intercepts and decrypts it with her own secret key
- Eve re-encrypts it with Bob's public key and forwards it to Bob

Public-key Certificates

- Bind one's identify with her/his public key
- Such binding is called a certificate
- The certificate is endorsed/guaranteed by a trusted third-party, the Certificate Authority
- The CA has a public key PK_{CA} and a secret SK_{CA}
- The CA can sign a certificate proving the binding of a public key and one identity, given that the CA has verified the identity of the requester
- The CA needs to publish its own certificate (self signed or signed by another trusted CA) to avoid interception

Public-key Certificate Example



***.google.com**

Issued by: Google Internet Authority G3

Expires: Tuesday, December 18, 2018 at 3:43:00 PM Hong Kong Standard Time

✓ This certificate is valid

▼ Details

Subject Name

Country US

State/Province California

Locality Mountain View

Organization Google LLC

Common Name *.google.com

Issuer Name

Country US

Organization Google Trust Services

Common Name Google Internet Authority G3

Serial Number 4848086134840224065

Version 3

Signature Algorithm SHA-256 with RSA Encryption (1.2.840.113549.1.1.11)

Parameters None

Not Valid Before Tuesday, September 25, 2018 at 3:43:00 PM Hong Kong Standard Time

Not Valid After Tuesday, December 18, 2018 at 3:43:00 PM Hong Kong Standard Time

Public Key Info

Algorithm Elliptic Curve Public Key (1.2.840.10045.2.1)

Parameters Elliptic Curve secp256r1 (1.2.840.10045.3.1.7)

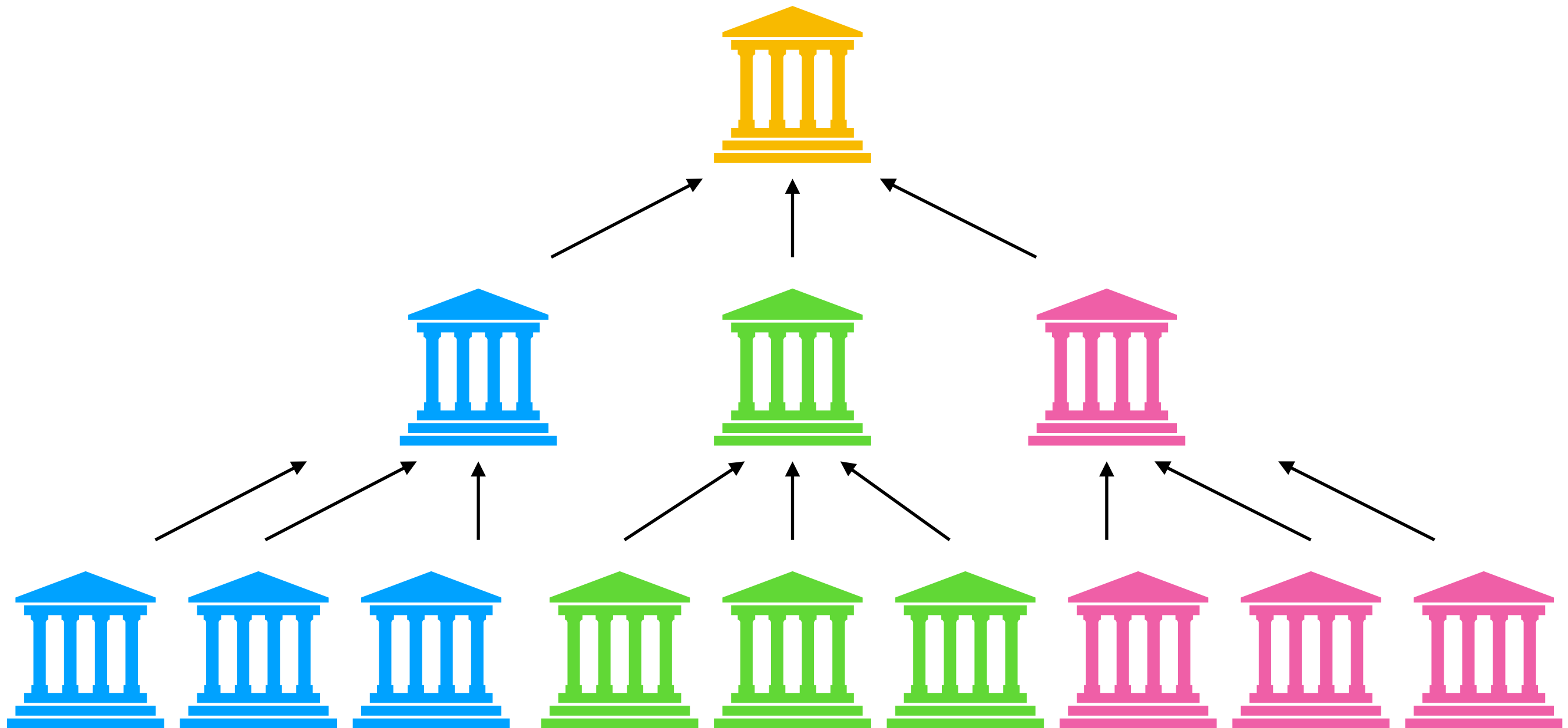
Public Key 65 bytes : 04 A0 42 C6 2B D3 E4 85 ...

Key Size 256 bits

Key Usage Encrypt, Verify, Derive

Signature 256 bytes : 51 3F 52 FC 77 2F B9 A4 ...

Public-Key Infrastructure



Credits

- Some slides are based on the slides of Vern Paxson, Dan Boneh, and Giovanni Vigna

OS Security

**CSCI 4130: Introduction to Computer
Security**

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

System Security

- Security properties
 - Confidentiality
 - Integrity
 - Authentication
- Some resources (e.g., files) are sensitive
- How do we protect them from an attacker?
 - How do we prohibit the attacker from accessing those resources?

Access Control

- Access control is the selective restriction of access to resources according to a policy
- Fundamentals
 - Subject (Principal): the entity who is accessing the resource
 - E.g., a user, a process, ...
 - Object: a resource that can be accessed
 - E.g., a file, a device, a web page,
 - Policy: how subjects can access on objects
 - E.g., Alice cannot read file X

Access Control (Cont.)

- Authorization
 - Who should be able to perform which actions
- Authentication
 - Identify who is requesting to perform the action
- Audit
 - Log all actions attributed to a particular principal
- Accountability
 - Hold a principal accountable for actions he/she takes

Access Control Policy

- $\text{access}(S, O) = \text{true}$
 - Subject S is allowed to access object O
- Examples
 - $\text{access}(\text{Alice}, \text{Alice's home directory}) = \text{true}$
 - $\text{access}(\text{Alice}, /etc/passwd) = \text{false}$
 - $\text{access}(\text{Bob}, \text{Alice's home directory}) = \text{false}$
 - $\text{access}(\text{root}, /etc/passwd) = \text{true}$
 - $\text{access}(\text{Alice}, /share/bob/f.txt) = \text{true}$

Access Control Matrix

- The access control policies can be managed in a matrix

	Alice's home directory	Bob's home directory	/etc/passwd	/share/bob/f.txt	...
Alice	TRUE	FALSE	FALSE	TRUE	
Bob	FALSE	TRUE	FALSE	TRUE	
root	TRUE	TRUE	TRUE	TRUE	
...					

Permissions

- We can have finer-grained permissions on access to an object
 - Read, Write, Execute
- Examples
 - `access(Alice, Alice's home directory) = {read, write}`
 - `access(Alice, /etc/passwd) = {}`
 - `access(Bob, Alice's home directory) = {}`
 - `access(root, /etc/passwd) = {read, write}`
 - `access(Alice, /share/bob/f.txt) = {read}`

Access Control Matrix (Cont.)

	Alice's home directory	Bob's home directory	/etc/passwd	/share/bob/f.txt	...
Alice	{R, W}	{}	{}	{R}	
Bob	{}	{R, W}	{}	{R, W}	
root	{R, W}	{R, W}	{R, W}	{R, W}	
...					

Security Mechanisms

- There are two primary security mechanisms for enforcing access control policies
- Access Control List (ACL)
 - Each object has a list of who has what kind of access to it
- Capability-based Security
 - Each subject has an unforgeable token that grants access

Access Control List

- An object centric mechanism
- Every object has an ACL that defines what access permissions each subject can have on the object
- Each access is checked against the object's ACL
 - Identify the subject accessing it
 - Find the permission the subject has on the ACL
 - Make access control decision
- Each column of the access control matrix is an access control list
- Example: guest list

Capability-based Security

- A subject centric mechanism
- A capability defines a value that references an object with an associated set of access permissions
- Each access is checked against the capability
 - Subject presents the capability
 - The reference monitor checks if the capability has the access permission
 - Make access control decision
- Example: key
 - Another one?

Reference Monitor

- A reference monitor is used to mediate all accesses to objects
- Desired properties
 - Unbypassable
 - Tamper-proof
 - Verifiable
- Subjects cannot access objects directly
 - All accesses must go through the reference monitor, which checks if an access should be granted or not

Trusted Computing Base

- The Trusted Computing Base (TCB) is the component of the system that has to be correct for some security goal to be achieved
- All trusts are built upon the correctness of the TCB
- Ideally, TCB should be unbypassable, tamper-proof and verifiable
- Example
 - The TCB for enforcing file access control includes the OS kernel and file system drivers

DAC vs. MAC

- Discretionary Access Control
 - Object owners can set the access control policy for the objects they own
 - Example: Linux
- Mandatory Access Control
 - Access control policies for all objects are set by a centralized authority
 - Example: SELinux

Role Based Access Control

- Access control matrix can grow complex as number of subjects, objects, and possible operation grow
- Observation: Users change more often than roles

	/hr	/eng	/marketing	/all
Marketing	{}	{}	{R,W}	{R}
Executive	{R}	{R}	{R, W}	{R, W}
Engineering	{}	{R, W}	{}	{R}
HR	{R, W}	{}	{}	{R}

Attribute Based Access Control

- Assign attributes to subjects and/or objects
- Design security policies using attributes
- Access control matrix defines which attributes the subjects need to access objects with certain attributes
- Example
 - Role Based Access Control

UNIX Security Model

- Subjects
 - Users
- Objects
 - Directories
 - Files
 - Other types of files
 - sockets, pipes, hardware devices, ...
- Operations
 - Read, Write, Execute

UNIX Groups

- A UNIX user may belong to one or more groups
- The groups can be used to provide role based access control
- A user belongs to
 - A primary group (defined in /etc/passwd)
 - Several additional groups (defined in /etc/group)
- Use the command **groups** to check your groups

UNIX File Access Control

- Every file and directory has an owner and a simple ACL
- The ACL specify what role can perform what actions
 - Three roles
 - Owner, Group, Other
 - Three operations
 - Read, Write, Execute
- Permissions are set by the owner (or root)
 - Use command **chmod**
 - Cannot be delegated
- Example: rwx r-x r-x

Sticky Bit

- Can a user rename a file in a directory where the user has only write permission on the directory but not the file?
- Sticky bit
 - Off
 - If user has write permission on directory, can rename and delete files, even if not owner
 - On
 - Only file owner, directory owner, or root can rename or delete the file

UNIX Process IDs

- Each process has a real UID/GID, an effective UID/GID, and a saved UID/GID
 - **Real IDs**: represents the user who starts/owns the process
 - **Effective IDs**: used to determine if the process is “allowed to do things”
 - **Saved IDs**: used to drop and re-gain privileges
- If an executable file has the SUID bit set, when a process executes this program the process’s effective IDs are changed to the ones of the program owner

```
wei@wei ~ >>> ls -la /bin/su
-rwsr-xr-x 1 root root 40536 May 17 2017 /bin/su
```

Linux Capabilities

- Traditionally Unix distinguished between privileged processes (EUID == 0) and unprivileged processes (EUID != 0)
- Privileged processes are exempt from all kernel permission checks, while unprivileged processes are subject to all permission checks
- Capabilities can divide the power of superuser into pieces
 - If a program with several capabilities is compromised, the damage it can do to the system would be much less than the same program running with root privilege

Privilege

- To limit the impact of a security compromise
 - **Separating** privileged operations into a small separate TCB
 - The TCB must be verified for correctness
 - Any security bugs outside the TCB will not be catastrophic
 - **Isolating** untrusted components from the rest of the system (privileged components)
 - Give the least privilege required to perform tasks

Sandbox

- A sandbox is a mechanism that isolates a process from the rest of the environment
- If some (vulnerable) code is from an untrusted source or takes inputs from potential attackers, running in a sandbox aims to prevent the exploits from spreading outside of the sandbox
- Various sandbox designs exist, but typically the sandbox
 - Provides a controlled set of resources for guest programs to run in (e.g., memory, disk)
 - Do not allow (all) network access, file system access, other I/O

Android Process Isolation

- Android Application Sandbox
 - **Isolation**: Each application runs with its own UID in its own VM
 - Memory protection
 - Communication limited to using unix domain sockets
 - Only ping, zygote run as root
 - **Interaction**: Reference monitor checks permissions on inter-component communication
 - **Least Privilege**: Application announces permission
 - User grants access at install time or request time

Chrome Security Model

- The modern chrome browser uses a multi-process architecture
 - The main browser process
 - Manage user data, address bar, settings
 - Privileged operations, e.g., network, file I/O, ...
 - Several renderer processes
 - Render remote content in each tab
 - Restricted permissions
 - Run in a sandbox

Chrome Security Model (Cont.)

- The unprivileged renderer process can request privileged operations from the browser process
- If a renderer process is crashed (because of a remote attack)
 - The damage is limited to only this process
 - No other site data is at risk
 - The main browser process cannot be compromised either

Information Flow Tracking

- A security mechanism to track how information (or data) propagates as a system/program computes
- Objects are labeled with tags, denoting security classes
 - Confidential data
 - Downloaded from Internet, e.g., browser or mail
 - ...
- The tags are updated and verified along the computation
- Can help reason about confidentiality and integrity properties

Credits

- Some slides are based on the slides of David Wagner, and Zakir Durumeric

Security Principles

CSCI 4130: Introduction to Cyber Security

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Security Principles

- Set of principle that help to design more secure software and systems
 - Keep the computer systems work as intended
 - Protect the data and computing resources from unauthorized use
- Keep them in mind when you develop any systems
- However, following the principles do NOT make your systems absolutely free from attacks

Knowing Your Threat Model

- What will a document safe protect against?
 - The unskilled burglar
 - The skilled burglar
 - None of the above
- Most “safes” are “fire safes” designed to prevent damage in case of a fire
- You need to know what you are protecting and what your threat is
- Perform threat modeling always **before** and **after** you deploy any security mechanism

Economy of Security Mechanism

- Increase the **cost or difficulty** for an **attacker** to attack, such that the **reward** the attacker gets is not enough to pay the cost
- As a **defender**, **costs/expenses** are also incurred to implement or deploy some protection mechanisms
- There is often a tradeoff between the **cost of security** and the **level of security** of a security mechanism
- Example:
 - A TL-15 safe will take 15 min to break using common tools and cost you about \$1,000
 - A TL-30 safe will take 30 min to break using common tools and cost you about \$2,000

Open Design

- The security evaluation in the previous example assumes an attacker with the proper tools and **knowledge of the safe's design**
- **DO NOT** rely on security through obscurity
 - Kerckhoffs's principle: “A cryptosystem should be secure even if everything about the system, except the key, is public knowledge”.
 - Shannon's maxim: “the enemy will immediately gain full familiarity with the systems”.
- It is easier to protect and revoke the key, than to keep the design of an algorithm/system secret
- Open design allows more people to examine the system and identify potential flaw in its design and implementation

Fail-safe Defaults

- Base the access decision on **permission** rather than **exclusion**
- Without explicit permission, the default action is deny of access
- Alternatively, a mechanism that attempts to identify conditions under which access should be refused, is not recommended
- Why?

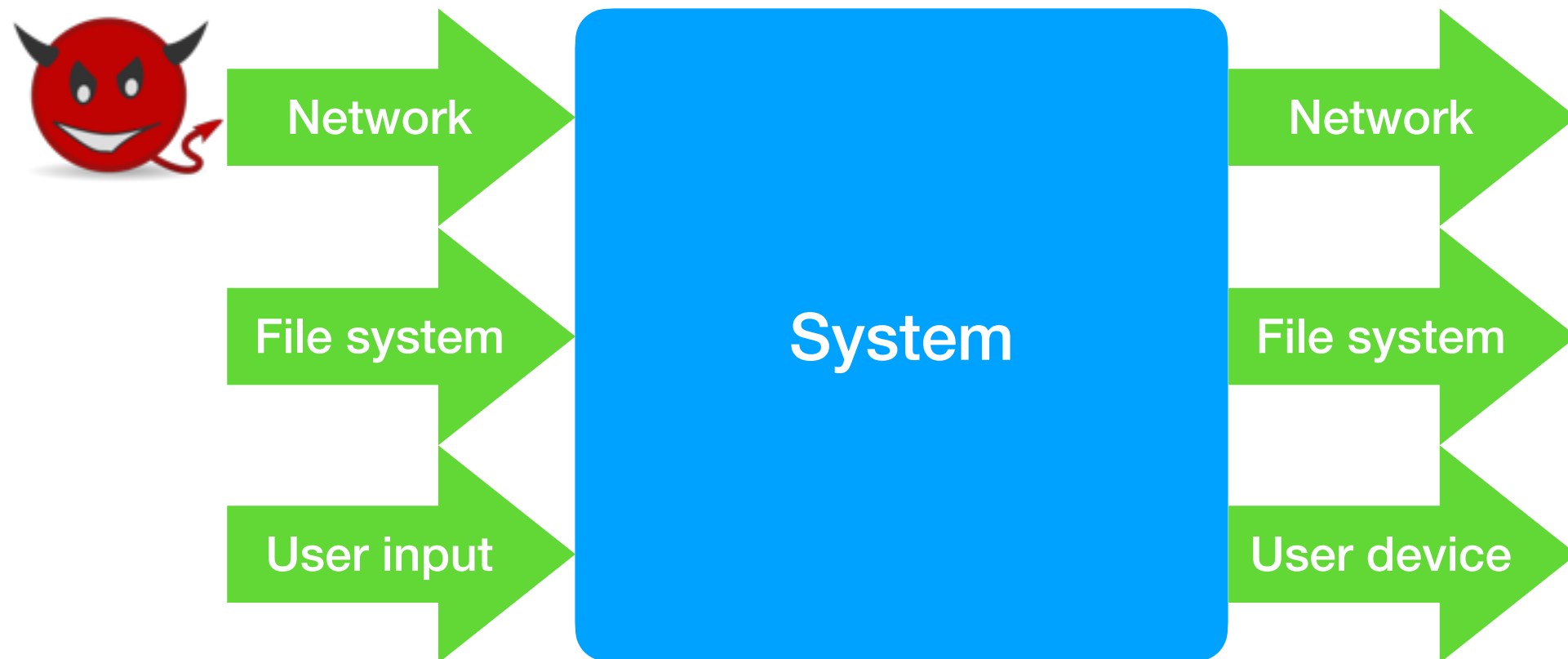
Least Privilege

- What **can** a BitTorrent client do?
 - Send your private files to someone else?
 - Delete all of your files?
 - Install a new executable?
- What **should** a BitTorrent client do?
 - Open and receive connections
 - Download and save files to some directories
 - Share those files with other peers

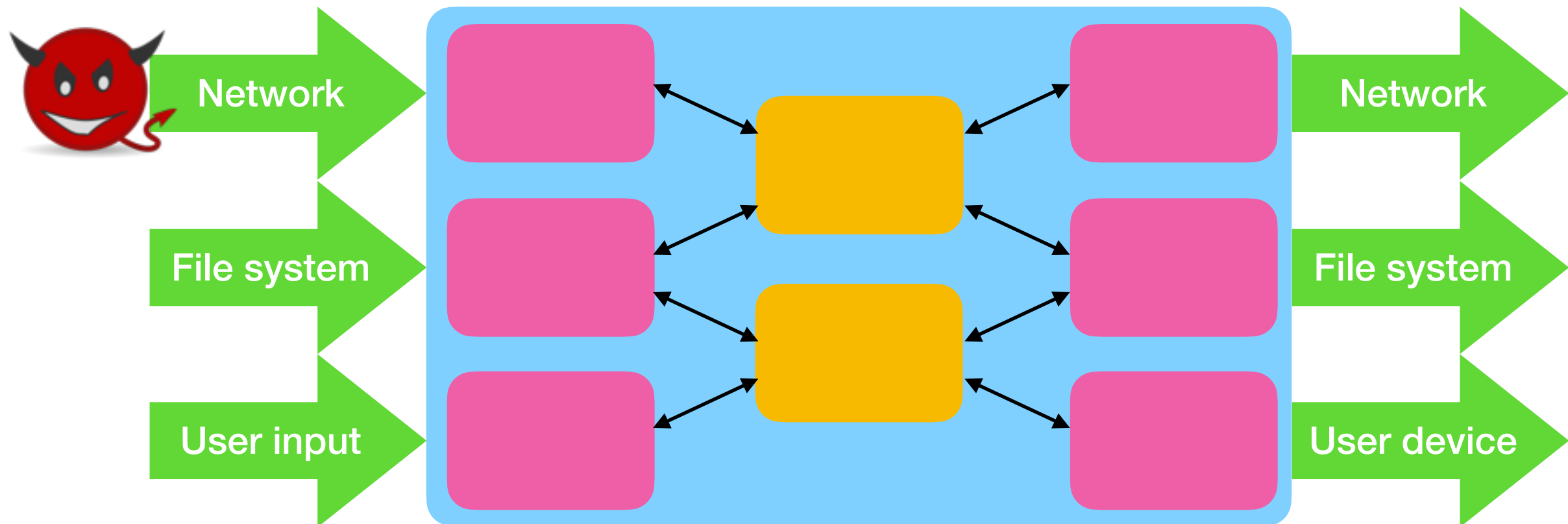
Least Privilege (Cont.)

- A system module should only have the minimal privileges needed for its intended purposes
- What is a privilege?
 - Ability to access a resource
- It assumes compartmentalization and isolation
 - Separate the system into isolated compartments
 - Each compartment is designed for one single purpose
 - Isolate the interaction between the compartments

Monolithic Design



Component Design



Complete Mediation

- Every access to every resource must be checked for authority



Complete Mediation (Cont.)

- To secure access to some resource, construct a **reference monitor**
- All accesses must go through the check of the reference monitor
- Example?
- Desired properties
 - Un-bypassable
 - Tamper-proof
 - Verifiable

Separation Of Privilege

- When feasible, separate the privilege required to perform an action



Defense In Depth

- If you use multiple redundant protections, then all of them would need to be breached before the system's security will be endangered
- When you use more than one security mechanism, what properties do you require them to have?
 - They should be as different as possible, so if one fails, it's unlikely that the second one fails (in the same way)
- What are other examples of defense in depth?

Defense In Depth (Cont.)

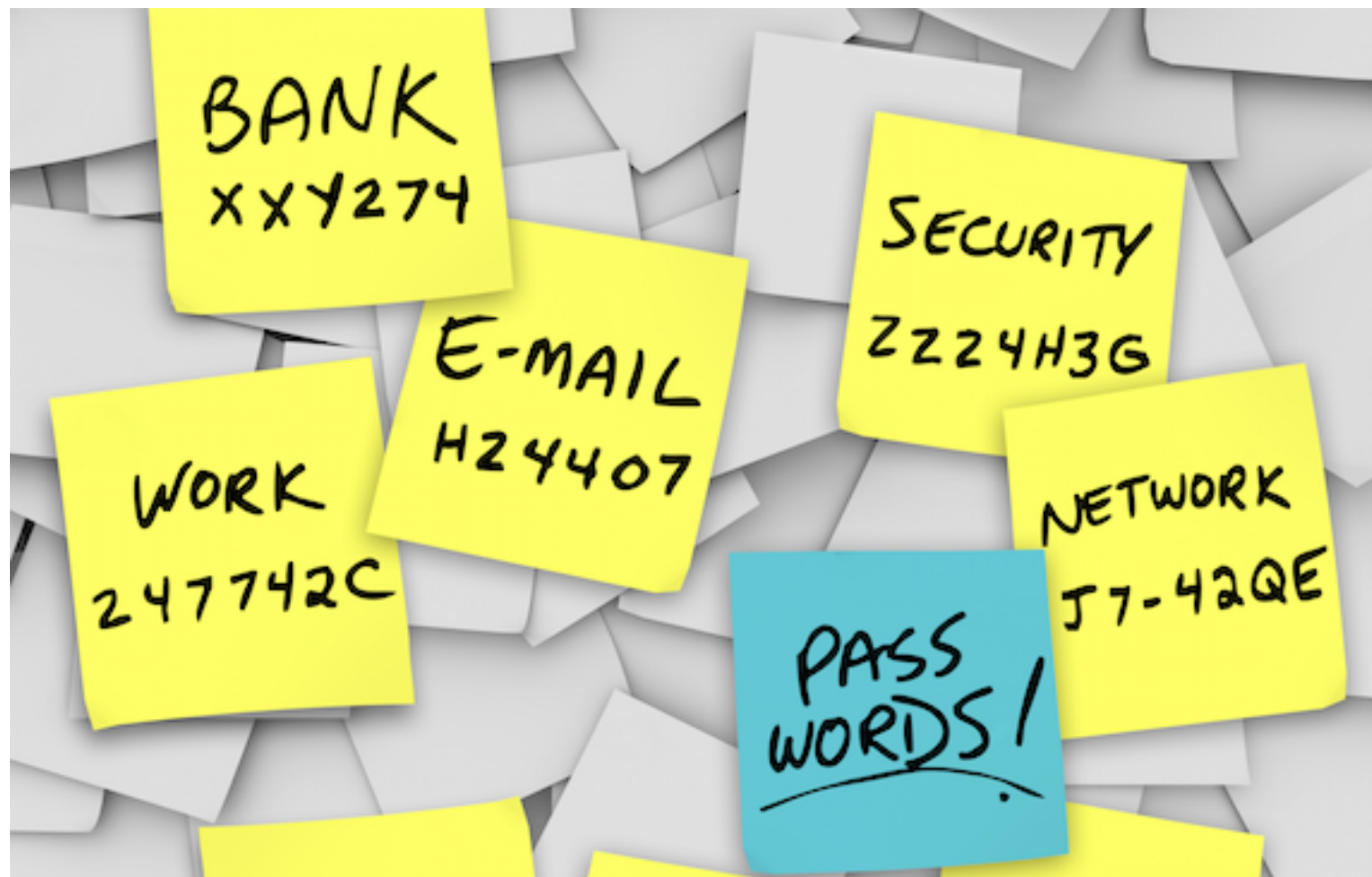
- Two-factor authentication
 - Authentication using two of:
 - Something you know
 - Something you have
 - Something you are
- Examples
 - Online banking
 - Password (sth you know) & Security token (sth you have)
 - Two-factor authentication w/ mobile phone
 - Password (sth you know) & SMS verification code (sth you have)

Defense In Depth (Cont.)

- Another example: a security mechanism requires a password and answers to several security questions
- Is it two-factor authentication?

Human Factors

- Company policy: passwords must be 10-20 characters long, contain at least 4 different characters, at least 1 uppercase character, 1 lowercase character, 1 digit, and 1 special character



Human Factors (Cont.)

“Humans are incapable of securely storing high-quality cryptographic keys, and they have unacceptable speed and accuracy when performing cryptographic operations. (They are also large, expensive to maintain, difficult to manage, and they pollute the environment. It is astonishing that these devices continue to be manufactured and deployed. But they are sufficiently pervasive that we must design our protocols around their limitations.)”

— Network Security: Private Communication in a Public World,
Charlie Kaufman, Radia Perlman, & Mike Speciner, 1995

- How to securely store a policy-compliant password?

Human Factors (Cont.)

Unable to verify the identify of www.facebook.com as a trusted site.

Possible reasons for this error:

- Your browser does not recognize the Certificate Authority that issued the site's certificate
- ...
- ...
- ☒ Accept this certificate permanently
- ☐ Accept this certificate temporarily for this session
- ☐ Do not accept this certificate and do not connect to this Web site

OK

Cancel

Human Factors (Cont.)

Unable to verify the identify of www.facebook.com as a trusted site.

Possible reasons for this error:

- Blah blah blah blah blah blah
- Blah blah blah blah
- ...

- ☒ Make this message disappear permanently
- ☐ Make this message disappear this time
- ☐ Block my visit and show me this dialog again next time

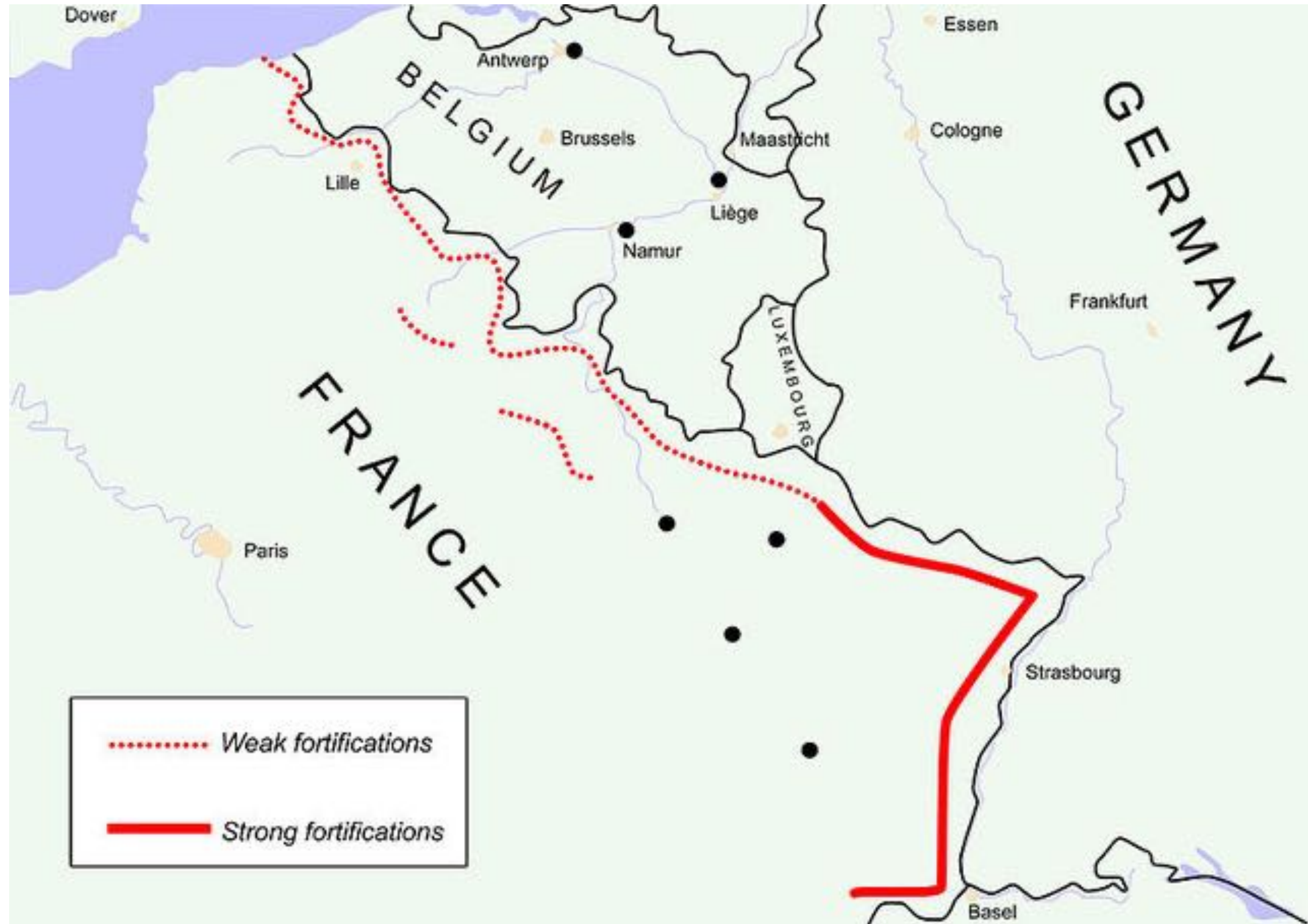
OK

Cancel

Dealing with Users

- Psychological acceptability
 - Will users abide a security mechanism, or decide to subvert it?
 - Make your system easy to use in a secure way
 - Make sure users understand how to use the system securely
- Human factors
 - Does a security mechanism assume something about human behavior when interacting with the system that might not hold, even in the absence of conscious decisions by the users to subvert

Only as Secure as the Weakest Link



Summary

- Knowing your threat model
- Economy of security mechanism
- Open design
- Fail-safe defaults
- Least privilege
- Complete mediation
- Separation of privilege
- Defense in depth
- Psychological acceptability
- Considering human factors
- Only as secure as the weakest Link

Credits

- Some slides are based on the slides of Vern Paxson, Raluca Ada Popa and John Mitchell